
Universidade de Aveiro
Departamento de Electrónica, Telecomunicações e Informática

**Gonçalo Fonseca, Marta Condeço, Pedro Martins,
Tiago Coelho e Vasco Pereira**

EduPro
Course Management and Evaluation System

Advisors - João Almeida, Vicente Barros

ABSTRACT

Keywords: Academic Assessment, Exam Management, Automated Grading, Question Bank

The management of academic assessments in higher education involves a complex, labor-intensive cycle of exam creation, administration, correction, and reporting. Existing tools address only isolated segments of this workflow, forcing educators to rely on a fragmented ecosystem of different solutions. This work presents EduPro, a web-based platform designed to unify the entire paper-based assessment lifecycle into a single workflow.

EduPro provides a structured, topic-organized question bank that supports both manual creation and bulk import of questions via Moodle-compatible XML files. An automated generation engine draws from this bank to produce multiple unique, typeset exam variants in PDF format, each including a QR code that uniquely identifies the exam. During the live exam session, a Web Application enables vigilantes to associate student identities with their respective exam through QR code scanning. Completed answer sheets are later processed by an Optical Mark Recognition pipeline that combines classical image processing techniques. The platform then provides a manual validation step before generating grade reports and dispatching individual results to students by email.

Access control is delegated by enforcing a role-based model that distinguishes between Managers, Regents, and Professors. The system is containerised using Docker and delivered through a fully automated CI/CD pipeline that integrates static analysis, integration testing, and automated deployment. Performance tests confirm that the platform sustains 50 concurrent users with a p95 response time below 140 ms and zero failures, meeting its defined non-functional requirements. Usability evaluation using the System Usability Scale yielded a score of 87.3.

The results demonstrate that EduPro is a viable, end-to-end solution for paper-based academic assessment management, reducing administrative overhead and improving the consistency and reliability of the evaluation process.

CONTENTS

ABSTRACT	i
LIST OF FIGURES	iv
LIST OF TABLES	vi
ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Context	1
1.2 Motivation	2
1.3 Goals	2
1.4 Document Structure	3
2 STATE OF THE ART	4
2.1 Literary Review of Assessment Challenges	4
2.1.1 Academic Integrity and Randomized Exams	5
2.2 Technological Alternatives and Competitor Analysis	5
2.2.1 Learning Management Systems (LMS)	5
2.2.2 Autograding Tools	6
2.2.3 Technical and Code-Centric Tools	7
2.2.4 Comparison and Gap Identification	7
3 INCEPTION	9
3.1 Personas	9
3.1.1 The Professors	9
3.1.2 The Managers	10
3.1.3 Relevance	10
3.2 Use Cases	11
3.2.1 Detailed Use Case Definitions	12
3.3 Functional Requirements	22
3.4 Non-Functional Requirements	23
3.5 Summary	24
4 IMPLEMENTATION	25
4.1 Architecture	25
4.2 Data Model	27
4.3 Exam Workflow	28
4.4 Backend	30
4.4.1 Core Domain Services	30
4.4.2 Domain Validation and State Integrity	31
4.4.3 Orchestration of Resource-Intensive Operations	31
4.5 Optical Mark Recognition	32
4.5.1 Overview	32
4.5.2 QR Code Detection	32
4.5.3 Grid Localisation and Perspective Correction	33
4.5.4 Cell Extraction	33

4.5.5	Evolution of the Optical Mark Recognition (OMR) Strategy	34
4.5.6	CNN Based Cell Classification	34
4.5.7	Scoring and Annotation	35
4.6	Web Application	35
4.6.1	Curricular Unit Management	37
4.6.2	Subject Administration	39
4.6.3	Frontend Technologies and Architecture	62
4.7	Deployment and Infrastructure	62
4.7.1	Infrastructure and Containerization	62
4.7.2	Development Workflow and Branch Protection	63
4.7.3	CI/CD Pipeline	64
4.7.4	Test Validation Strategy	65
4.8	Security	66
4.8.1	Authentication and Authorization	66
4.8.2	Data Validation and Rate Limiting	67
4.8.3	Injection Prevention	68
4.8.4	Security Hardening and Fixes	68
4.9	Summary	68
5	RESULTS AND DISCUSSION	70
5.1	Deployment	70
5.2	Performance Testing	70
5.3	Usability Evaluation	71
5.3.1	Methodology	71
5.3.2	Results	72
5.4	Discussion	72
6	CONCLUSION	73
6.1	Main Results	73
6.2	Retrospective	74
6.3	Future Work	74
	REFERENCES	75
	ANNEX A — USE CASE DIAGRAM	76

LIST OF FIGURES

2.1	Moodle sandbox.	6
4.1	System Architecture Diagram	26
4.2	EduPro Relational Data Model	27
4.3	Exam Session State Machine and Primary Transitions	29
4.4	Example of a generated exam sheet variant with unique identification.	31
4.5	Multi-pass Quick Response (QR) Code Detection on an Answer Sheet	33
4.6	Grid Cell Extraction and Coordinate Mapping	34
4.7	EduPro landing page in light and dark mode.	36
4.8	Keycloak registration form	36
4.9	Keycloak login form and error feedback.	36
4.10	Curricular units page before any unit is created.	37
4.11	Curricular unit list with existing units.	37
4.12	Modal dialog for creating a new curricular unit.	38
4.13	Modal dialog for editing an existing curricular unit.	38
4.14	Delete action selected on a curricular unit.	39
4.15	Deletion confirmation dialog.	39
4.16	Curricular units page for non-manager roles.	40
4.17	Subject detail view as seen by a regent.	40
4.18	Professor assignment form before any selection.	41
4.19	Professor list updated after confirmation.	41
4.20	Import hint tooltip	42
4.21	Preview of parsed questions before import confirmation.	42
4.22	Question bank after bulk import.	43
4.23	Topic creation modal.	43
4.24	Newly created topic expanded in the question bank.	44
4.25	Question creation modal filled and ready for submission.	44
4.26	Inline validation feedback when question constraints are not met.	45
4.27	Question added to the bank after creation.	45
4.28	Topic name edit modal with updated name.	46
4.29	Question bank showing the topic with its updated name.	46
4.30	Edit question modal with updated content.	47
4.31	Updated question in the bank after editing.	47
4.32	Two-step deletion of a question from the bank.	48
4.33	Two-step deletion of a topic from the question bank.	48
4.34	Exams tab before any session has been created.	48
4.35	Exam creation step 1: topic selection.	49
4.36	Exam creation step 2: number of questions per topic.	49
4.37	Exam creation step 3: topic weights and wrong-answer penalty.	50
4.38	Exam creation step 4: session name and date.	50
4.39	Exam creation step 5: Comma-Separated Values (CSV) format tooltip.	51
4.40	Exam creation step 5: vigilants selected and students imported.	51
4.41	Exam creation final step: full parameter summary before submission.	52
4.42	Exam card transitioning from processing to concluded after generation.	52
4.43	Session detail page showing the eight-step stepper.	53
4.44	Step 1 expanded: configuration summary and exam package download.	53
4.45	Starting the exam session on desktop and mobile.	54

4.46	Student-exam paper association on desktop and mobile.	55
4.47	Closing the exam session on desktop and mobile.	56
4.48	Answer sheet capture and upload on desktop and mobile.	57
4.49	Step 6: conflict resolution interface for association anomalies.	58
4.50	Per-exam manual review and validation in Step 7.	58
4.51	Step 8: email composition with content toggles and live preview.	59
4.52	Student email: grade only.	59
4.53	Student email: grade and colour-coded answer grid.	60
4.54	Student email: answer grid and question weights table.	60
4.55	Student email: all options enabled including cumulative score table.	61
4.56	Exported grades at the end of the assessment lifecycle.	61
4.57	GitHub Tags and Releases	63
4.58	GitHub Pull Request Workflow	64
4.59	Main Branch Automated Checks	65
4.60	SonarCloud Analysis Report	66
4.61	Authentication and Authorisation Sequence Flow	67
4.62	Data validation and mapping using Pydantic schemas.	67
1	Use Case Diagram for EduPro.	76

LIST OF TABLES

2.1	Comparative analysis of proposed solution and its main competitors.	7
3.1	Overview of use cases per persona.	11
3.2	Overview of functional requirements.	23
5.1	K6 load testing results for two virtual user scenarios.	71

ABBREVIATIONS

API	Application Programming Interface
OCR	Optical Character Recognition
UI	User Interface
QR	Quick Response
PDF	Portable Document Format
SUS	System Usability Scale
RBAC	Role-Based Access Control
LMS	Learning Management System
OMR	Optical Mark Recognition
SQL	Structured Query Language
SPA	Single-Page Application
CI	Continuous Integration
CD	Continuous Deployment
JWT	JSON Web Token
CNN	Convolutional Neural Network
PWA	Progressive Web Application
TLS	Transport Layer Security
URL	Uniform Resource Locator
CSV	Comma-Separated Values
XML	Extensible Markup Language
DTD	Document Type Definition

CHAPTER 1

INTRODUCTION

Assessment is a fundamental component of the higher education ecosystem, serving as the primary metric for student performance and comprehension. For professors and academic departments, the logistics of managing courses, organizing curricula into specific topics, and ensuring fair, rigorous evaluation is a continuous challenge. As class sizes grow, the administrative burden of manually creating exams, preventing academic dishonesty through variant generation, and grading assessments increases exponentially. Traditionally, these tasks have been performed using disjointed tools, leading to inefficiencies, data redundancy, and a higher potential for human error. EduPro emerges as an integrated platform designed to bridge these gaps by unifying exam preparation, automated grading, and content management into a single, cohesive workflow. This chapter outlines the context and motivations for the project, its core objectives, and the structure of this document.

1.1 CONTEXT

In the contemporary landscape of higher education, the management of pedagogical content and student assessment has become increasingly complex. Academic institutions are tasked with delivering high-quality education to growing student populations while maintaining rigorous standards of evaluation. Assessment is a continuous cycle that includes the planning of curricular units, the organization of knowledge into structured topics, and the creation of various evaluation instruments such as exams and quizzes.

Currently, professors and educational coordinators rely on a fragmented ecosystem of tools to manage these responsibilities. Learning Management System (LMS) like Moodle provide a foundation for digital learning, yet many administrative tasks, such as the generation of unique physical exam variants, the maintenance of structured question banks, and the manual grading of multiple-choice tests, remain heavily dependent on traditional office software and manual effort. This traditional approach often fails to scale effectively with the demands of modern academic environments, where consistency, security, and efficiency are paramount.

1.2 MOTIVATION

The primary motivation for this project stems from the significant inefficiencies and risks inherent in the manual management of the assessment lifecycle. The process of preparing exams, particularly for large classes, is often a labor-intensive endeavor that requires professors to manually select questions, format documents, and create multiple variants to ensure academic integrity. This fragmented approach consumes a significant amount of time and introduces a high potential for human error, ranging from inconsistent formatting to mistakes in manual grading.

Furthermore, the lack of a centralised and structured repository for educational content leads to “information silos”, where exams exist in many different formats. This makes it difficult to standardise automatic correction over heterogeneous exams. There is a clear need for a solution that can automate the mechanical aspects of exam generation and grading, leveraging technologies such as [OMR](#) and structured content management. By transforming these fragmented, manual processes into a unified and automated digital workflow, EduPro aims to reduce administrative overhead, enhance the accuracy of assessments, and provide educators with better tools for curricular management.

1.3 GOALS

The primary objective of this project is to design and implement EduPro, a web-based platform dedicated to the management of academic assessments and curricular content. In order to achieve this, the project sets out to accomplish the following specific goals:

- **Structured Question Bank:** The foundation of the platform is a question bank that allows for the categorisation, storage, and retrieval of assessment items across different topics and difficulty levels, providing the organised pool from which all exams are drawn.
- **Automated Exam Generation:** Building on this repository, an exam generation engine produces unique variants from the question bank based on specific configurations — such as the number of questions per topic and randomisation strategies — and outputs them in high-quality Portable Document Format ([PDF](#)) format.
- **Unique Identification and Tracking:** Each generated paper is assigned a [QR](#) code that uniquely identifies it, enabling seamless association between students and their specific exam variants and supporting reliable traceability throughout the assessment process.
- **Automated Grading via [OMR](#):** Once completed papers are scanned, an [OMR](#) system automatically detects and evaluates student answers, significantly reducing the manual grading effort required from educators.
- **Integrated Communication and Reporting:** Finally, the platform closes the assessment lifecycle by automatically generating performance reports and notifying students of their results via email.

Together, these goals aim to deliver a cohesive platform that modernises the academic assessment workflow and reduces the administrative burden on educators.

1.4 DOCUMENT STRUCTURE

This document is organised into six chapters that progressively develop the EduPro platform from conception to evaluation. Following this introduction, Chapter 2 explores the current landscape of educational technology, reviewing existing platforms and technologies for exam management and automated grading to identify gaps and opportunities that motivate the proposed approach.

Chapter 3 details the theoretical design of the system, establishing the architectural patterns, the data models representing the “Subject-Topic-Question” hierarchy, and the design of the OMR engine that underpin the subsequent implementation.

Chapter 4 then describes the technical development of the platform, covering the selection of technologies — React, FastAPI, and PostgreSQL — the integration with Keycloak for authentication, and the challenges encountered during the development of the OMR and L^AT_EX generation engines.

Chapter 5 presents the functional prototype and evaluates its performance against the initial requirements, discussing the accuracy of the OMR system and the usability of the platform.

Finally, Chapter 6 concludes the dissertation by summarising the project’s contributions, reflecting on its current limitations, and outlining directions for future development and scaling. The report also includes appendices containing supplementary material such as detailed diagrams and code snippets.

CHAPTER 2

STATE OF THE ART

This chapter provides a comprehensive review of the current landscape in educational assessment management. Following a structured approach, the first phase explores the common challenges in university assessment. The second phase examines current technological solutions and competitors, identifying the specific gaps in the market.

2.1 LITERARY REVIEW OF ASSESSMENT CHALLENGES

The process of student assessment is a critical yet resource-intensive component of higher education. Academic environments face growing pressures due to increasing class sizes and the need for more frequent, high-stakes evaluations. For a professor managing a course with hundreds of students, the assessment cycle from exam creation to final grade submission represents a significant administrative burden that competes directly with teaching and research responsibilities. A study by Mandernach and Holbeck found that faculty spend approximately 40% of their total instructional time on grading and feedback, amounting to roughly five hours per week per course [1]. This proportion illustrates how assessment-related tasks dominate the workload of educators, leaving less time for activities with direct pedagogical impact.

Manual grading is a particular bottleneck in this workflow. When a professor must correct dozens or hundreds of answer sheets sequentially, the cumulative cognitive load leads to fatigue, causing a gradual decline in consistency and attention as the volume of work increases. Beyond errors caused by fatigue, manual grading of multiple-choice exams is also simply slow: each sheet must be checked question by question against an answer key, a task that scales linearly with the number of students and offers no opportunity for parallelism or automation in a traditional setting.

2.1.1 ACADEMIC INTEGRITY AND RANDOMIZED EXAMS

One of the core requirements of EduPro is the ability to generate multiple distinct exam variants from the same question bank. This requirement is directly motivated by the need to preserve academic integrity in supervised examination environments. Maintaining academic integrity in physical vigilation environments has its challenges. Randomizing exam versions (shuffling questions and options) is a proven strategy [2] to deter cheating. However, implementing this manually for large groups is complex and prone to human error. Logistical hurdles include the difficulty of printing multiple distinct versions, ensuring correct distribution in an exam hall, and the administrative overhead of mapping randomized answer keys back to individual students.

2.2 TECHNOLOGICAL ALTERNATIVES AND COMPETITOR ANALYSIS

Several platforms and tools currently exist to assist educators in different parts of the assessment lifecycle. During the research phase of this project, these tools were tested and evaluated in the context of higher education institutions where formal examinations are conducted on paper. This is a widespread practice that introduces requirements around physical document generation, distribution, and correction that purely digital platforms are not designed to address.

2.2.1 LEARNING MANAGEMENT SYSTEMS (LMS)

[LMS](#) platforms are the most widely adopted category of educational software and represent the most natural starting point for any institution looking to digitise its assessment workflow. Their course management and question bank features overlap directly with EduPro’s scope, making them the primary point of comparison.

Moodle is a widely used open-source [LMS](#) that excels at course management and online quizzing. During examination, it is possible to create a structured question bank and configure randomized online assessments with relative ease. However, because formal examinations in many institutions are still conducted on paper rather than digitally, Moodle’s online-first design becomes a fundamental limitation. Generating randomized [PDF](#) versions for physical printing is not a native feature: it requires the installation and configuration of third-party plugins such as [qbank_genai](#)¹, making the workflow fragmented and inaccessible for many users. For institutions where paper-based exams are the norm, Moodle covers only part of the assessment lifecycle, as seen in the sandbox environment shown in Figure 2.1.

Open edX is a platform primarily designed for Massive Open Online Courses (MOOCs), offering robust automated grading for digital submissions². Its evaluation workflow is built

¹[Moodle – Generative AI Question Bank](#)

²[Open edX – Create an Open Response Assessment](#)

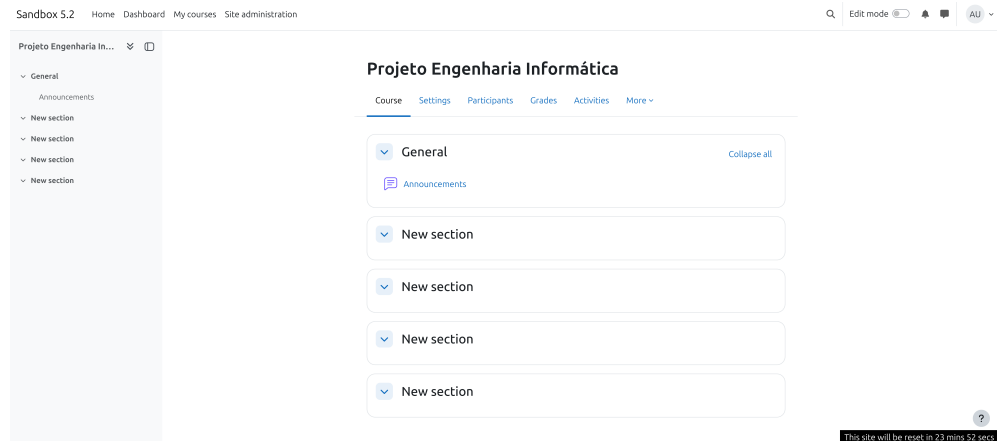


FIGURE 2.1 – Moodle sandbox.

around digital-first interaction, with strong support for open response assessments and peer review. For institutions that conduct formal examinations on paper, Open edX offers no viable pathway: it provides no mechanism for generating printable exam documents or processing completed answer sheets. The platform is well-suited for online learning environments but was not designed with physical assessment workflows in mind.

2.2.2 AUTOGRADING TOOLS

To incorporate automated grading of physical answer sheets into an integrated assessment platform, it is essential to first analyse the capabilities and limitations of existing dedicated autograding solutions. This research phase focused on identifying where these tools excel and where they fail to provide a complete exam management workflow.

ZipGrade³ is a mobile-centric application that enables automated grading of custom answer sheets via a smartphone camera. It is highly efficient for quick, on-the-spot grading and requires minimal setup. Despite its speed advantages, ZipGrade operates in isolation: it does not manage any course structure or “Subject-Topic-Question” hierarchy, nor does it generate the exam papers themselves. Users must rely entirely on separate tools to author and print the exams, making ZipGrade only one piece of a larger, still-fragmented puzzle.

ExamIO⁴ and PhotoExamAI⁵ represent a newer generation of platforms focused on automated grading of scanned answer sheets with high accuracy. These tools are capable of processing multiple-choice sheets without specialized hardware, which lowers the barrier to adoption. However, they function as standalone, “one-off” grading utilities. They lack integration with a persistent question bank or course management system, making it impossible to track student progress across semesters or reuse assessment content in a structured way.

³ZipGrade – Cloud-based Grading Service

⁴ExamIO – Automatic Grading for Tests and Exams

⁵PhotoExamAI – AI-powered Exam Grading

2.2.3 TECHNICAL AND CODE-CENTRIC TOOLS

Beyond [LMS](#) and autograding platforms, tools targeting technical education are also worth considering, as they show how automated grading pipelines can work end-to-end in a different domain.

GitHub Classroom⁶ and nbgrader⁷ are prominent tools within computer science and engineering education. GitHub Classroom integrates with version control workflows to collect and manage programming assignments, while nbgrader enables the automated grading of Jupyter notebooks. Both tools excel in their domain and provide a largely automated feedback loop for technical coursework. Their fundamental limitation, however, is that they are highly domain-specific: they offer no support for the physical, paper-based exams that remain the standard evaluation method for many theoretical and non-technical subjects in higher education.

2.2.4 COMPARISON AND GAP IDENTIFICATION

The table [2.1](#) summarizes the feature set of the evaluated competitors compared to the proposed EduPro system.

TABLE 2.1 – Comparative analysis of proposed solution and its main competitors.

Feature	EduPro	Moodle	Open edX	ZipGrade
Course & Question Bank Mgmt.	Yes	Yes	Yes	No
Randomized PDF Exam Gen.	Yes	Limited	No	No
Physical Exam Workflow	Yes	No	No	No
Student-Exam Association (QR)	Yes	No	No	No
Automated Answer Sheet Grading	Yes	No	No	Yes
Grade Export & Notification	Yes	Yes	Partial	No
Unified End-to-End Workflow	Yes	No	No	No

The analysis reveals that existing tools address isolated parts of the assessment lifecycle but none provides a unified solution for the full paper-based exam workflow. [LMS](#) platforms such as Moodle and Open edX are strong in course and content management but were designed

⁶GitHub Classroom – Manage assignments and help students

⁷nbgrader – A tool for grading Jupyter notebooks

for digital assessments and lack support for generating physical exam variants, managing vigilance sessions, or processing completed answer sheets. Autograding tools like ZipGrade handle answer sheet correction efficiently but operate entirely independently from any course structure or exam generation pipeline. EduPro’s key differentiator is that it is the only platform covering the entire lifecycle: from question bank management and randomized [PDF](#) generation, through QR-code-based student association during the exam session, to automated grading, grade export, and student notification.

CHAPTER 3

INCEPTION

This chapter presents the conceptual model of the system, establishing a shared understanding of its structure, behaviour, and boundaries before any implementation decisions are made. Through a set of complementary modelling techniques, it captures the functional requirements and the interactions between the system and its stakeholders, serving as the foundation for subsequent design and development phases.

3.1 PERSONAS

The academic assessment cycle is often perceived as a series of isolated events, a lecture, an exam, a grade. However, for those behind the scenes, it is a complex and frequently chaotic logistical operation. During peak assessment periods, university departments transform into hubs of manual labor, where professors struggle with stacks of paper and managers navigate fragmented spreadsheets. The pressure to maintain academic integrity while managing growing student populations often leads to burnout and a high margin for human error. To understand the necessity of a unified solution, we must look at the daily challenges faced by the individuals who anchor this process.

The main stakeholders in this ecosystem can be represented by two major groups: the professors, who are on the front lines of teaching and evaluation, and the managers, who orchestrate the administrative framework of the institution.

3.1.1 THE PROFESSORS

Consider professor Helena, a dedicated regent for a first-year Computer Science course with over 250 students. For Helena, exam season is synonymous with exhaustion. To prevent academic dishonesty in a crowded amphitheater, she must manually design three different versions of the same exam, ensuring they are equivalent in difficulty but distinct enough to

discourage copying. This process involves days of meticulous formatting in traditional text editors, where a single typo in a question can invalidate dozens of papers.

Following the administration of the exam, the most time consuming phase begins: the grading. Helena and her team of teaching assistants spend two weeks of late nights manually marking multiple-choice sheets and calculating final scores. The fear of a simple summation error, one that could unfairly change a student's final classification, is a constant source of anxiety. For Helena, the problem is not the teaching; it's the administrative wall that stands between her and her students' actual progress.

3.1.2 THE MANAGERS

On the other side of the department is Ricardo, the Academic Coordinator. Ricardo does not grade exams, but he is responsible for the "administrative scaffolding" that allows the department to function. He manages multiple curricular units, ensuring that every course has a designated regent and an assigned team of professors.

Currently, Ricardo's world is one of constant coordination and fragmented communication. At the start of every semester, he navigates a sea of emails and spreadsheets to map multiple professors to their respective units. When a change in regency occurs or a new professor joins the staff, the update process is manual and prone to being overlooked across various disjointed lists. He often finds himself answering repeated inquiries about who is responsible for which course. For Ricardo, the lack of a centralized management tool means that maintaining an accurate institutional registry is a constant struggle against administrative entropy. He needs a system that formalizes these assignments, ensuring that the right people are empowered with the necessary permissions from day one.

3.1.3 RELEVANCE

The proposed system optimizes the educator's workflow by automating repetitive administrative tasks and enhancing the objectivity of student assessments. By automating the mechanical aspects of exam generation and grading, she can focus on the pedagogical quality of her questions rather than the logistics of their distribution. For Ricardo, the platform provides a centralized source of truth for the department's organisation. It eliminates the ambiguity of staff assignments and ensures that regents are instantly empowered with the administrative tools they need to manage their courses. Ultimately, this project is relevant because it addresses the human cost of academic administration, replacing manual frustration with digital precision.

3.2 USE CASES

This section describes the system’s functionality from the perspective of its actors, detailing the interactions they can perform and the expected outcomes. Each use case represents a discrete unit of behaviour that the system must support, providing a clear and traceable link between stakeholder needs and system requirements. An overview of these use cases, organised by persona, is presented in Table 3.1.

TABLE 3.1 – Overview of use cases per persona.

Persona	Use Case
Manager	UC-1 Create a new curricular unit. UC-2 Delete a curricular unit. UC-3 Change the regent of a curricular unit.
Regent	UC-4 Create a topic to a curricular unit’s question bank. UC-5 Delete a topic from a curricular unit’s question bank. UC-6 Edit a topic’s name. UC-7 Create a question within a topic. UC-8 Delete a question from a topic. UC-9 Edit a question’s title and/or answer(s). UC-10 Import topic(s) and/or question(s) via XML file upload. UC-11 Generate an exam. UC-12 Delete an exam. UC-13 Add a professor to the vigilant teachers’ list. UC-14 Remove a professor from the vigilant teachers’ list. UC-15 Open an exam. UC-16 Associate students to exams. UC-17 Close an exam. UC-18 Take a photo and send it to be corrected. UC-19 Resolve exam warnings. UC-20 View an exam’s auto correction. UC-21 Edit an exam’s auto correction. UC-22 Validate an exam’s correction. UC-23 Export exams’ grades to a PDF file. UC-24 Notify all students via email.
Professor	UC-25 Act as vigilant and associate students to exams
Manager / Regent	UC-26 Add a professor to a curricular unit. UC-27 Remove a professor from a curricular unit.
Regent / Professor	UC-28 Associate a student to an exam via QR code scanning.
Everyone	UC-29 View a curricular unit’s regent and professors’ list.

A visual representation of the use cases and their relationships can be found in the UML Use Case Diagram in 6.3.

3.2.1 DETAILED USE CASE DEFINITIONS

This section provides formal specifications for the core use cases of the platform, detailing the steps, actors, and conditions required for their successful execution.

UC-1: CREATE A NEW CURRICULAR UNIT

Description: To establish a new curricular unit within the department's academic registry.

Actors: Manager.

Pre-conditions: The Manager is authenticated in the system.

Post-conditions: A new curricular unit is persisted in the database and visible in the department's list.

Scenario

1. **Actor:** Navigates to the curricular unit management section and selects the option to create a new unit.
2. **System:** Displays a form requesting the unit's name, code, and academic year.
3. **Actor:** Enters the required details and submits the form.
4. **System:** Validates the information, saves the new unit, and displays a success confirmation.

UC-2: DELETE A CURRICULAR UNIT

Description: To permanently remove a curricular unit and its associated data from the system.

Actors: Manager.

Pre-conditions: The Manager is authenticated; the curricular unit exists in the database.

Post-conditions: The curricular unit is removed from the database and no longer appears in the registry.

Scenario

1. **Actor:** Selects a specific curricular unit from the management list and chooses the delete option.
2. **System:** Displays a warning message asking for confirmation of the permanent deletion.
3. **Actor:** Confirms the action.
4. **System:** Deletes the record and refreshes the list to show the updated registry.

UC-3: CHANGE THE REGENT OF A CURRICULAR UNIT

Description: To update the professor responsible for the management of a specific curricular unit.

Actors: Manager.

Pre-conditions: The Manager is authenticated; the curricular unit and the target professor exist.

Post-conditions: The regent role is reassigned, updating the permissions for both the old and new regents.

Scenario

1. **Actor:** Accesses the staff management page for a specific curricular unit.
2. **System:** Shows the current staff list and highlights the current regent.
3. **Actor:** Selects a different professor from the list to be the new regent.
4. **System:** Updates the database, reassigns the regency permissions, and confirms the change.

UC-4: CREATE A TOPIC TO A CURRICULAR UNIT'S QUESTION BANK

Description: To add a new category for organising questions within a curricular unit's question bank.

Actors: Regent.

Pre-conditions: The Regent is authenticated and assigned to the specific curricular unit.

Post-conditions: A new topic is created and available for question assignment.

Scenario

1. **Actor:** Selects the curricular unit and navigates to the "Question Bank" section.
2. **System:** Displays the current topics and an option to add a new one.
3. **Actor:** Enters the name for the new topic and confirms.
4. **System:** Persists the new topic and updates the view to include it.

UC-5: DELETE A TOPIC FROM A CURRICULAR UNIT'S QUESTION BANK

Description: To remove an existing topic from the question bank.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the topic exists and is preferably empty.

Post-conditions: The topic is removed from the question bank.

Scenario

1. **Actor:** Identifies the topic to be removed within the question bank list and selects the delete option.
2. **System:** Requests confirmation for the deletion.
3. **Actor:** Confirms the deletion.
4. **System:** Removes the topic from the database and updates the display.

UC-6: EDIT A TOPIC'S NAME

Description: To modify the label of an existing topic for better organisation or clarity.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the topic exists.

Post-conditions: The topic's name is updated in the database and reflected in the UI.

Scenario

1. **Actor:** Selects the edit option for a specific topic in the question bank.
2. **System:** Displays an input field with the current name of the topic.
3. **Actor:** Enters the new name and saves the changes.
4. **System:** Updates the topic's record and confirms the successful modification.

UC-7: CREATE A QUESTION WITHIN A TOPIC

Description: To add a new assessment item (question and its options) to a specific topic.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the target topic exists.

Post-conditions: A new question with its associated options is persisted in the database.

Scenario

1. **Actor:** Selects a topic and chooses the option to add a new question.
2. **System:** Displays a form for the question title, content, difficulty level, and multiple-choice options.
3. **Actor:** Fills in the question details, marks the correct option(s), and submits.
4. **System:** Validates the input, saves the question and options, and links them to the topic.

UC-8: DELETE A QUESTION FROM A TOPIC

Description: To permanently remove a specific question and its related options from the system.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the question exists.

Post-conditions: The question and its options are removed from the database.

Scenario

1. **Actor:** Locates the question within a topic and selects the delete action.
2. **System:** Displays a confirmation dialog warning about permanent deletion.
3. **Actor:** Confirms the deletion.
4. **System:** Deletes the question and its options, refreshing the topic's question list.

UC-9: EDIT A QUESTION'S TITLE AND/OR ANSWER(S)

Description: To update the content, metadata, or correct answers of an existing question.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the question exists.

Post-conditions: The question's data is updated in the database.

Scenario

1. **Actor:** Selects the edit option for a specific question.

2. **System:** Loads the current question data into an editable form.
3. **Actor:** Modifies the title, text, or the correctness of the options and saves.
4. **System:** Validates and updates the record, providing a confirmation message.

UC-10: IMPORT TOPIC(S) AND/OR QUESTION(S) VIA XML FILE UPLOAD

Description: To bulk-populate the question bank using a structured [XML](#) file.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the [XML](#) file follows the required schema.

Post-conditions: Multiple topics and questions are created and stored in the database.

Scenario

1. **Actor:** Selects the import option in the question bank management section.
2. **System:** Displays a file upload dialog.
3. **Actor:** Selects and uploads the [XML](#) file.
4. **System:** Parses the [XML](#), validates the content against the schema, imports the data, and reports the number of items successfully added.

UC-11: GENERATE AN EXAM

Description: To create unique exam variants in [PDF](#) format based on question bank content and specific configuration.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the question bank contains sufficient questions for the requested configuration.

Post-conditions: One or more unique [PDF](#) exam files are generated and stored; tracking [QR](#) codes are associated with the variants.

Scenario

1. **Actor:** Navigates to the exam generation section and selects "Generate New Exam".
2. **System:** Displays configuration options such as the number of questions per topic, number of variants, and randomization settings.
3. **Actor:** Configures the exam parameters and initiates the generation process.
4. **System:** Randomizes questions, generates LaTeX documents, compiles them into [PDF](#)s with unique [QR](#) codes, and confirms successful generation.

UC-12: DELETE AN EXAM

Description: To remove a generated exam and its associated variants and tracking data from the system.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the exam exists.

Post-conditions: Exam records and associated [PDF](#) files are removed from the system.

Scenario

1. **Actor:** Selects an exam from the list of generated assessments and chooses to delete it.
2. **System:** Displays a confirmation prompt emphasizing that all associated data and files will be lost.
3. **Actor:** Confirms the deletion.
4. **System:** Removes the exam records and physical PDF files from storage and updates the list.

UC-13: ADD A PROFESSOR TO THE VIGILANT TEACHERS' LIST

Description: To authorize a professor to act as a vigilant for a specific exam session.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the exam exists; the target professor is enrolled in the curricular unit.

Post-conditions: The professor is added to the exam's vigilant list and granted the necessary permissions for student association.

Scenario

1. **Actor:** Accesses the vigilant management settings for a specific exam.
2. **System:** Displays the list of professors associated with the curricular unit who are not yet vigilants for this exam.
3. **Actor:** Selects one or more professors to add as vigilants.
4. **System:** Updates the exam's vigilant list, assigns permissions to the selected professors, and confirms the action.

UC-14: REMOVE A PROFESSOR FROM THE VIGILANT TEACHERS' LIST

Description: To revoke a professor's authorization to act as a vigilant for a specific exam.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the professor is currently on the vigilant list.

Post-conditions: The professor is removed from the list and their vigilant-specific permissions for the exam are revoked.

Scenario

1. **Actor:** Navigates to the vigilant list of a specific exam.
2. **System:** Shows the current vigilants.
3. **Actor:** Selects a professor and chooses to remove them.
4. **System:** Updates the records, revokes permissions, and confirms the removal.

UC-15: OPEN AN EXAM

Description: To transition an exam into the active state, allowing vigilants to begin associating students with their papers.

Actors: Regent.

Pre-conditions: The Regent is authenticated; the exam has been generated.

Post-conditions: The exam state is set to "Open", enabling the association features for authorized staff.

Scenario

1. **Actor:** Selects a generated exam that is currently in a "Closed" or "Draft" state.
2. **System:** Displays the option to "Open" the exam for administration.
3. **Actor:** Confirms the opening of the exam.
4. **System:** Updates the exam state in the database, enables association interfaces for vigilants, and confirms the status change.

UC-16: ASSOCIATE STUDENTS TO EXAMS

Description: To link a specific student to their unique exam variant using identification data.

Actors: Regent, Professor (Vigilant).

Pre-conditions: The exam is in the "Open" state; the student and exam variant exist in the system.

Post-conditions: The student's identity is linked to the specific exam paper in the database.

Scenario

1. **Actor:** Selects an exam variant (via manual entry or [QR](#) scan) and inputs the student's identification.
2. **System:** Validates the student information and the exam variant's availability.
3. **Actor:** Confirms the association between the student and the paper.
4. **System:** Saves the link in the database and displays a success confirmation.

UC-17: CLOSE AN EXAM

Description: To transition an exam session to a "Closed" state, preventing further student associations and marking it ready for correction.

Actors: Regent.

Pre-conditions: The exam is in the "Open" state.

Post-conditions: The exam state is "Closed"; association features are disabled for all staff.

Scenario

1. **Actor:** Selects the active exam and chooses the "Close" option.
2. **System:** Displays a confirmation prompt showing the total number of students associated.
3. **Actor:** Confirms the action.
4. **System:** Updates the exam state to "Closed", disables vigilant association tools, and confirms the status change.

UC-18: TAKE A PHOTO AND SEND IT TO BE CORRECTED

Description: To upload a digital image of a completed exam sheet for automated processing by the [OMR](#) engine.

Actors: Regent.

Pre-conditions: The exam is in the "Closed" state; the student paper has been previously associated.

Post-conditions: The image is uploaded, and the automated correction process is initiated.

Scenario

1. **Actor:** Navigates to the correction section for an exam and selects a specific student paper.
2. **System:** Opens the camera interface or a file upload dialog.
3. **Actor:** Captures or selects the image of the completed exam sheet and submits it.
4. **System:** Uploads the image to storage, triggers the [OMR](#) service for analysis, and confirms receipt.

UC-19: RESOLVE EXAM WARNINGS

Description: To manually review and correct anomalies identified by the [OMR](#) system during automated grading.

Actors: Regent.

Pre-conditions: The [OMR](#) process has completed with warnings (e.g., ambiguous marks or missing fields).

Post-conditions: Warnings are cleared, and the grading data is corrected based on manual review.

Scenario

1. **Actor:** Accesses the "Warnings" dashboard for a specific exam.
2. **System:** Displays a list of detected issues alongside the original image for reference.
3. **Actor:** Selects a warning, reviews the image, and manually overrides or confirms the system's detection.
4. **System:** Updates the record, clears the warning, and recalculates the final grade if necessary.

UC-20: VIEW AN EXAM'S AUTO CORRECTION

Description: To inspect the grades and marked choices generated by the automated [OMR](#) system for a specific student paper.

Actors: Regent.

Pre-conditions: The [OMR](#) process for the specific paper has completed successfully.

Post-conditions: The Regent is presented with the system's findings and the preliminary grade.

Scenario

1. **Actor:** Selects a corrected student paper from the exam results list.
2. **System:** Displays the analyzed image with visual overlays showing detected marks and the calculated score.
3. **Actor:** Reviews the detected answers and the associated points.
4. **System:** Maintains the results display and offers options for manual validation.

UC-21: EDIT AN EXAM'S AUTO CORRECTION

Description: To manually adjust the system-detected answers or scores for a specific student paper.

Actors: Regent.

Pre-conditions: The [OMR](#) process has completed; the Regent has viewed the results.

Post-conditions: The student's recorded answers and final grade are updated based on manual changes.

Scenario

1. **Actor:** Selects the "Edit" option while viewing a student's auto-correction results.
2. **System:** Enables interactive fields to modify the detected choices for each question.
3. **Actor:** Updates the choices based on a visual review of the original exam sheet.
4. **System:** Recalculates the grade in real-time, saves the changes, and logs the manual override.

UC-22: VALIDATE AN EXAM'S CORRECTION

Description: To finalize the grading process for a student paper, marking the score as official.

Actors: Regent.

Pre-conditions: The [OMR](#) process has finished and any warnings have been resolved or reviewed.

Post-conditions: The correction status is set to "Validated," preventing further automated changes.

Scenario

1. **Actor:** Reviews the final results for a student paper and selects the "Validate" option.
2. **System:** Prompts for final confirmation of the grade.
3. **Actor:** Confirms the validation.
4. **System:** Updates the paper's status to "Validated" and locks the record for official reporting.

UC-23: EXPORT EXAMS' GRADES TO A **pdf!** ([pdf!](#)) FILE

Description: To generate a structured [PDF](#) document containing the final list of students and their respective grades.

Actors: Regent.

Pre-conditions: At least one student paper has been validated.

Post-conditions: A [PDF](#) report is generated and available for download.

Scenario

1. **Actor:** Navigates to the exam reporting section and selects the "Export Grades" option.
2. **System:** Compiles the validated scores into a formal grading list document.
3. **Actor:** Downloads the generated [PDF](#) file.
4. **System:** Serves the file and logs the export action.

UC-24: NOTIFY ALL STUDENTS VIA EMAIL

Description: To automatically send individual emails to students informing them of their final exam results.

Actors: Regent.

Pre-conditions: The exam session is closed and results have been validated.

Post-conditions: Emails are dispatched to all associated students with their specific score and feedback.

Scenario

1. **Actor:** Selects the option to Notify Students from the exam management dashboard.
2. **System:** Displays a summary of students to be notified and a preview of the email template.
3. **Actor:** Confirms the mass notification.
4. **System:** Dispatches the emails via the mail service and provides a status report of the delivery process.

UC-25: ACT AS VIGILANT AND ASSOCIATE STUDENTS TO EXAMS

Description: To perform the student-to-exam association task during an active exam session as an authorized vigilant.

Actors: Professor (Vigilant).

Pre-conditions: The Professor has been added to the exam's vigilant list; the exam is in the "Open" state.

Post-conditions: Student identities are linked to specific exam variants in the database.

Scenario

1. **Actor:** Accesses the Vigilant Dashboard for the assigned exam.
2. **System:** Displays the association interface with variant selection and student ID input.
3. **Actor:** Scans the [QR](#) code on a student's paper and enters their identification number.
4. **System:** Validates the data, creates the association, and confirms the success to the vigilant.

UC-26: ADD A PROFESSOR TO A CURRICULAR UNIT

Description: To associate an existing professor with a specific curricular unit, allowing them to participate in its management or administration.

Actors: Manager, Regent.

Pre-conditions: The Actor is authenticated and authorized; the curricular unit and the professor exist.

Post-conditions: The professor is linked to the curricular unit and appears in its staff list.

Scenario

1. **Actor:** Accesses the staff management section of a curricular unit.
2. **System:** Displays the current list of associated professors and an option to add a new one.
3. **Actor:** Searches for the professor by name or email and selects them.
4. **System:** Validates the selection, updates the association in the database, and confirms success.

UC-27: REMOVE A PROFESSOR FROM A CURRICULAR UNIT

Description: To revoke a professor's association with a curricular unit, removing their access and permissions for that specific course.

Actors: Manager, Regent.

Pre-conditions: The Actor is authorized; the professor is currently associated with the curricular unit.

Post-conditions: The association is removed from the database.

Scenario

1. **Actor:** Navigates to the curricular unit's staff list.
2. **System:** Shows all professors currently assigned to the unit.
3. **Actor:** Selects a professor and chooses the "Remove" action.
4. **System:** Displays a confirmation prompt.
5. **Actor:** Confirms the removal.
6. **System:** Deletes the association, revokes permissions, and updates the list.

UC-28: ASSOCIATE A STUDENT TO AN EXAM VIA **qr! (qr!) CODE SCANNING**

Description: To streamline the identification process by using the unique **QR** code on an exam paper to link it to a student.

Actors: Regent, Professor (Vigilant).

Pre-conditions: The exam is in the "Open" state; the Actor has access to a device with a camera.

Post-conditions: The scanned exam variant is associated with the student's ID.

Scenario

1. **Actor:** Activates the scanner within the student association interface.
2. **System:** Requests and receives access to the device's camera.
3. **Actor:** Positions the exam's QR code in front of the camera.
4. **System:** Scans the code, identifies the exam variant, and prompts for the student's identification.
5. **Actor:** Enters the student's ID and confirms.
6. **System:** Links the student to the paper and displays a success notification.

UC-29: VIEW A CURRICULAR UNIT'S REGENT AND PROFESSORS' LIST

Description: To check the current teaching staff and regency assignment for a specific course.

Actors: Everyone.

Pre-conditions: The user is authenticated.

Post-conditions: The user has viewed the staff list for the curricular unit.

Scenario

1. **Actor:** Navigates to the details page of a specific curricular unit.
2. **System:** Displays general information about the unit.
3. **Actor:** Selects the "Staff" or "Professors" tab.
4. **System:** Displays the name and role (Regent or Professor) of all personnel associated with the unit.

3.3 FUNCTIONAL REQUIREMENTS

This section details the functional requirements of the system, defining the specific actions and features that the platform must provide to support the academic assessment lifecycle. A consolidated overview of these requirements, including the primary personas associated with each, is presented in Table 3.2.

TABLE 3.2 – Overview of functional requirements.

ID	Persona	Requirement Description
FR-1	Manager	Create curricular units.
FR-2		Edit curricular units.
FR-3		Delete curricular units.
FR-4	Manager / Regent	Manage staff assignments for curricular units.
FR-5	Regent	Import/Create questions and topics in the question bank.
FR-6		Configure and generate unique exam variants.
FR-7		Manage exam session states (Open/Close).
FR-8		Resolve association warnings and anomalies.
FR-9		Correct and validate automated OMR results.
FR-10		Export finalized grades and notify students via email.
FR-11	Regent / Professor	Associate student identities with specific exam papers.
FR-12	Professor	Access curricular unit details and staff lists.

3.4 NON-FUNCTIONAL REQUIREMENTS

The following list presents the non-functional requirements, which specify the properties, characteristics, and constraints that the software system must exhibit.

- **Security:** The system must strictly protect user credentials by storing them using robust encryption standards, ensuring that unauthorized access to sensitive account information is prevented even in the event of a database compromise.
- **Performance:** To ensure consistent service quality and prevent system degradation, the infrastructure must be capable of handling a minimum of 50 concurrent users without significant latency or service interruption.
- **Usability:** The user interface and interaction design must be validated through user testing, requiring the system to achieve a score of at least 80 on the System Usability Scale (SUS), reflecting a high level of user satisfaction and ease of use.
- **Reliability:** In the event of an operational failure or invalid user input, the system must properly display clear, informative error messages. This ensures the user understands the nature of the issue and is not led to misunderstandings or frustration.
- **Authorization:** To ensure that data is accessed only by permitted personnel, the system must implement a Role-Based Access Control (RBAC) system, a security approach that restricts system access to authorized users based on their specific roles within the organisation. This ensures a clear definition of the permissions and features associated with each user level, such as Manager, Regent, or Professor.
- **Compatibility:** The web application must be accessible and fully functional across most modern web browsers, ensuring a consistent and responsive experience regardless of the user's preferred browsing environment.

3.5 SUMMARY

This chapter established the conceptual foundation for the EduPro platform by translating human challenges into technical specifications. Through the definition of key personas, we identified the critical pain points in academic administration, while the comprehensive mapping of use cases provided a detailed blueprint for the system's interactive behavior. The functional and non-functional requirements detailed here ensure that the project is not only feature-complete but also adheres to high standards of security, performance, and usability. Together, these elements form a rigorous framework that guides the architectural decisions and implementation strategies described in the following chapters, ensuring the final platform remains aligned with the needs of its stakeholders.

CHAPTER 4

IMPLEMENTATION

Following the conceptual framework and use cases defined in the previous chapter, this chapter describes the technical realization of the EduPro platform. It details the architectural decisions, the integration of specialized technologies, and the infrastructure strategies that bridge the gap between academic requirements and a functional digital ecosystem.

This chapter is organised into several key sections. It begins by presenting the system's topology and the layered separation between the presentation and business layers. Subsequently, the focus shifts to the underlying data structures and the complex state machine that governs the assessment lifecycle. The technical implementation of the backend services and the [OMR](#) processing pipeline are then explored in detail. Finally, the chapter discusses the frontend application architecture, followed by the deployment infrastructure and the multi-layered security framework that ensures the platform's integrity and reliability.

4.1 ARCHITECTURE

The technical architecture of EduPro, as illustrated in [Figure 4.1](#), was designed following a three-tier pattern, ensuring a clear separation of concerns between the presentation, business logic, and data management layers. This modular approach allows for greater flexibility, easier maintenance, and the ability to scale individual components independently as the platform's requirements evolve.

The presentation layer was developed using the React framework. This choice was driven by the need to create a highly flexible, responsive, and user-friendly interface that could seamlessly adapt between desktop-heavy administrative tasks and mobile-first classroom interactions. By leveraging a component-based architecture, we ensured a consistent user experience across the entire platform while maintaining a performant and interactive User Interface ([UI](#)).

For the business layer, the system utilizes FastAPI, a modern Python web framework. FastAPI was selected because it combines high development speed with strong technical

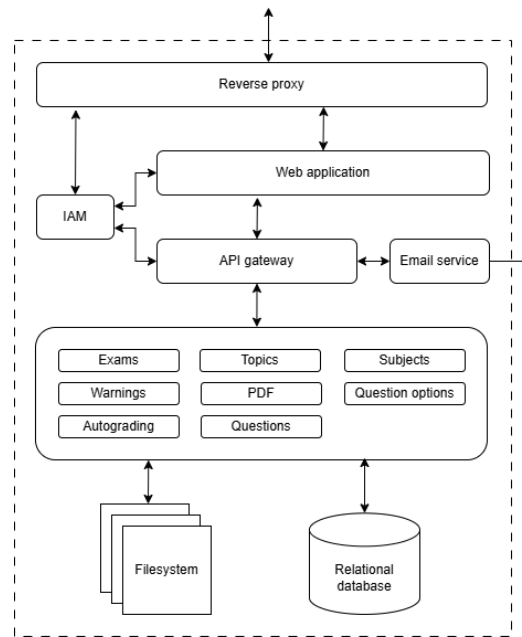


FIGURE 4.1 – System Architecture Diagram

consistency. Its use of Python type hints enables automatic data validation and generates comprehensive Application Programming Interface ([API](#)) documentation, which was critical for ensuring a robust contract between the frontend and the backend services. This layer orchestrates the core logic of the system, including the [OMR](#) processing and the exam generation engine.

The data layer is anchored by a PostgreSQL relational database. PostgreSQL was chosen for its proven reliability and support for complex relationships required by the Subject-Topic-Question hierarchy. To interface with this layer, we employed `SQLModel`, which allows us to define our data structures in Python and automatically synchronize them with the database schema, ensuring type safety from the database up to the [API](#) endpoints.

To handle the authentication and authorisation components, the system integrates Keycloak as a dedicated identity provider. Rather than implementing a custom security solution, we chose Keycloak to simplify the development process by leveraging a mature, fully developed product. This allows the platform to offload sensitive tasks, such as credential storage, session management, and [RBAC](#), to a secure and industry-standard service.

In accordance with the illustrated topology, the entire application was developed and deployed using a Docker container system. This containerisation strategy ensures that the application behaves consistently across development, testing, and production environments, simplifying the deployment process and allowing the various tiers of the architecture to run in a cohesive and isolated environment.

4.2 DATA MODEL

The relational data model, depicted in Figure 4.2, represents the core entities and their relationships within the EduPro platform. The relational schema is centred on the following entities, all defined as SQLModel classes.

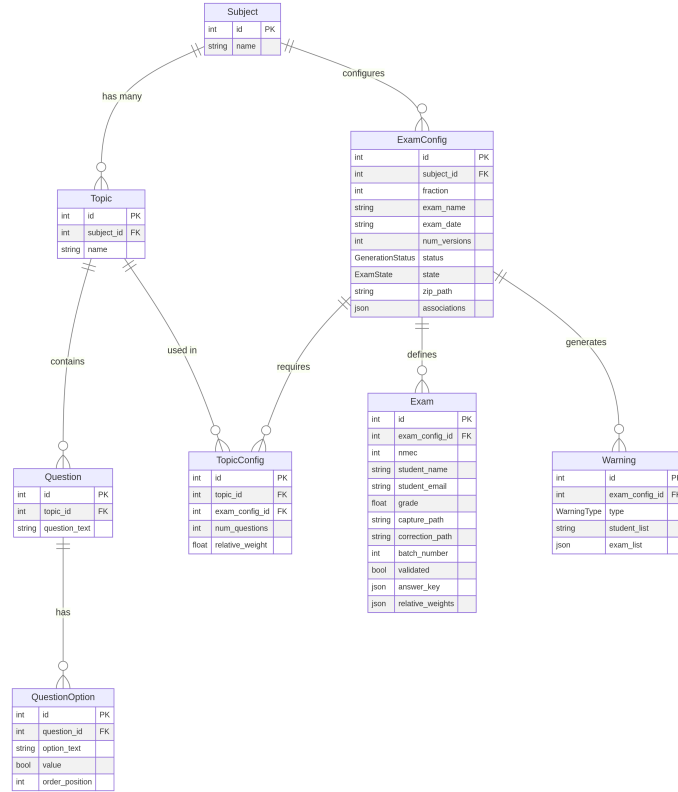


FIGURE 4.2 – EduPro Relational Data Model

The relational schema is organised around a hierarchical structure that facilitates the automated generation and processing of academic assessments. At the base of this hierarchy is the Subject, which represents a curricular unit and serves as the primary container for all related data and permissions. Access to these units is controlled through an integration with Keycloak, where specific groups designate the roles and editing capabilities of different users. Each subject is further divided into multiple Topics, which act as thematic categories to organise the pool of available questions.

The question bank itself consists of Questions and their related QuestionOptions. These entities represent multiple-choice items where specific options are marked as correct to enable automated grading. This structure allows the system to support diverse input methods, including manual entry through the interface and bulk imports from existing question banks. The separation between questions and their thematic topics ensures that the database can scale and remain organised as the volume of content increases.

When a new assessment is initiated, the system utilizes the ExamConfig entity as a central blueprint. This record defines the global parameters of the exam session, such as the penalty

applied to wrong answers and the metadata regarding the exam's name and date. The specific rules for creating individual booklets are defined through TopicConfig records, which specify how many questions are drawn from each topic and their relative contribution to the final grade. This configuration acts as a bridge between the static question bank and the dynamic generation of unique exam variants.

The result of the generation process is a set of Exam records, each corresponding to a physical booklet produced for a student. These records track the entire lifecycle of an assessment, including the randomized answer key, the student identification associated with the booklet via QR codes, and the final results extracted through the OMR engine. To maintain the reliability of the system, the Warning entity is used to document and flag any logical conflicts or anomalies discovered during the processing phase, such as booklets associated with multiple students or processed images that do not have a valid student link.

4.3 EXAM WORKFLOW

The exam workflow orchestrates the transition from a static question bank to a finalized and graded assessment. This lifecycle is managed through a unified state machine that ensures each phase is validated before the next one begins. The progression of an exam session is not merely a linear sequence of events, it involves a complex interaction between professors, vigilants, and automated services, encompassing both standard operation paths and exceptional cases that require manual intervention. The lifecycle is initiated by the definition of a structural blueprint, a phase that precedes the formal state machine transitions. This blueprint encompasses the core parameters of the assessment, including the selection of thematic topics, the required number of questions, their relative weights, and the enrollment list of participating students. The submission of this blueprint triggers the automated creation of the exam configuration and the generation of randomized booklets.

Upon successful generation of the materials, the session enters the preparing state. At this point, the initial configuration is considered complete, and the primary interaction shifts to administrative setup. In the preparing state, the professor can download the finalized exam package for printing and distribution, and the only remaining adjustment permitted is the selection of vigilantes. These users are assigned the necessary permissions to perform student associations via the mobile interface once the exam transitions to the running state, as illustrated in the state transitions depicted in Figure 4.3.

Once the exam is marked as running, the workflow shifts to the live execution phase. This stage is characterised by the real-time association of students to physical booklets using the mobile interface. A key aspect of this phase is support for non-linear interactions: vigilants can associate booklets in any order, and the system provides live feedback on the progress of associations. This flexibility is necessary in a physical examination setting where students may arrive at different times or require different versions of the exam.

The processing of the exam begins after the physical booklets are collected and pho-

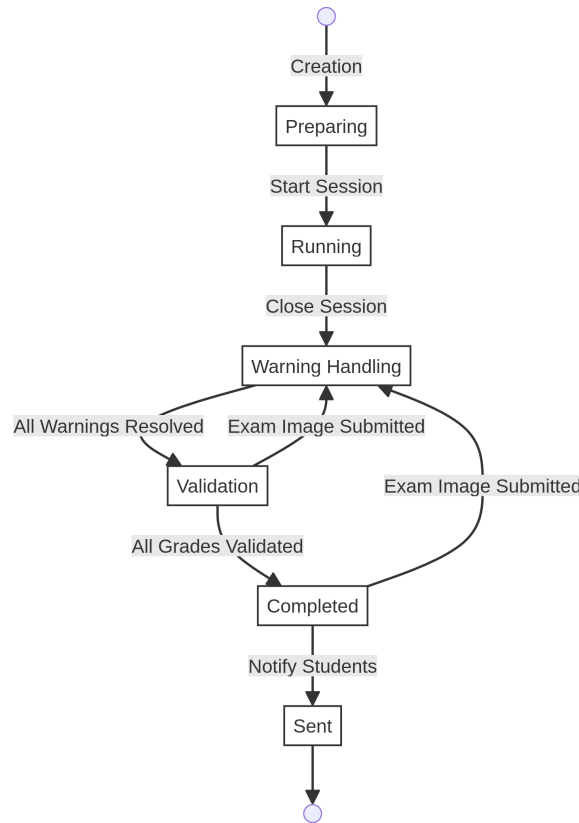


FIGURE 4.3 – Exam Session State Machine and Primary Transitions

tographed. The system moves into a phase dedicated to anomaly resolution, referred to as the warning handling state. This is where the workflow addresses exceptions that deviate from the normal processing path. The system may detect that a single booklet has been associated with multiple students due to vigilant error, or that a processed image belongs to a student who was never associated with a booklet. These anomalies are persisted as warning records that must be resolved manually by the professor.

This state machine has a recursive nature regarding image submissions. Even if a session has progressed to the validation or completed states, the submission of a new exam image via the [OMR](#) engine will automatically trigger a transition back to the warning handling state. This ensures that any new data is subjected to the same rigorous conflict detection as the initial batch. The system only permits forward movement to the validation phase once the internal logic confirms that the warning count has dropped to zero.

The final segments of the workflow involve validation and reporting. In the validation state, the system presents the automatically computed results for a final human review. This allows professors to correct any potential misreadings from the [OMR](#) engine, such as ambiguous marks or erasures. Only after every exam in the session has been validated can the state transition to completed. While in the completed state, the results are staged for release, but the system still remains open to new image submissions which would trigger a return to the warning handling phase. The lifecycle of the assessment only reaches its definitive conclusion in the sent state, which is triggered after the system facilitates the final distribution of grades

via automated email notifications.

4.4 BACKEND

The backend of EduPro is designed as a service-oriented system that encapsulates the platform’s core business domains. This section details the functional organisation of these services, the validation logic that maintains system integrity, and the orchestration of resource-intensive background operations.

4.4.1 CORE DOMAIN SERVICES

The business logic of the platform is partitioned into modular services, each responsible for a specific domain of the academic assessment lifecycle. This separation ensures that the system can manage diverse responsibilities, from simple administrative registries to complex document generation, within a unified framework.

The Curricular Unit Service manages the foundational registry of subjects and teaching teams. It translates institutional roles into granular system permissions, ensuring that only authorized regents can modify a course’s structure. Complementing this, the Question Bank Service facilitates the organisation of educational content into thematic topics. This service is responsible for parsing and validating bulk data imports, such as Moodle [XML](#), ensuring that imported questions meet the formatting requirements for subsequent LaTeX generation.

The Exam Orchestration Service is the most complex component of the backend, managing the structural configuration of assessments. It handles the random selection of questions based on topic-specific weights and constraints, ensuring that every generated variant is unique yet equivalent in difficulty. An example of a generated exam sheet, including the header and [QR](#) code tracking, is shown in [Figure 4.4](#).

Finally, the Correction and Warning Service acts as the bridge between the automated [OMR](#) results and the physical reality of the classroom. It monitors for logical conflicts, such as duplicate student associations or missing identification data, and generates actionable warnings for the professor.

Projeto em Engenharia Informática
2º Semestre, 2025/26

Versão 1
Exame Final
12 de junho de 2026

Número mecanográfico: _____
Nome: _____

O exame tem 20 perguntas de escolha múltipla.
As perguntas de escolha múltipla têm apenas uma resposta correta, devem ser respondidas na grelha presente nesta página do enunciado com um **X**. Para anular uma resposta, o aluno deve preencher **completamente** a célula. As respostas anuladas não descontam, mas as erradas **descontam** (cotação da pergunta $\times 0.35$).
Se uma célula de resposta ficar inutilizável (por exemplo, se a preencheu completamente para a anular, mas mais tarde decide que é a sua resposta final), indique a opção correta no espaço em redor da grelha de respostas. No entanto, isto deve ser feito **estritamente** dentro da linha pivotada.
A duração total do exame é de **1h30**.

*The exam has 20 multiple-choice questions.
The multiple-choice questions have only one correct answer and should be answered in the grid provided on this page of the exam with an **X**. To cancel an answer, the student should **completely** fill in the cell. Canceled answers do not result in deductions, but incorrect answers **deduct** (question score $\times 0.35$).
If an answer cell becomes unusable (for example, if you completely filled it in to cancel it but later decide it is your final answer), please write the correct option in the space around the answer grid. However, this must be done **strictly** within the dotted line.
The total duration of the exam is **1h30**.*



	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A																				
B																				
C																				
D																				

1. (0.40 pts) Which Spring module is responsible for handling web requests and creating RESTful APIs?
 A. Spring Security
 B. Spring Batch
 C. Spring Web MVC
 D. Spring Data JPA
2. (1.20 pts) Which file is central to a Maven project and contains all its configuration details?
 A. pom.xml
 B. build.gradle
 C. settings.xml
 D. package.json
3. (1.20 pts) What does the 'mvn clean' command do?

FIGURE 4.4 – Example of a generated exam sheet variant with unique identification.

4.4.2 DOMAIN VALIDATION AND STATE INTEGRITY

To ensure the reliability of the assessment process, the backend enforces a multi-layered validation strategy. These validations are deep business rules that prevent the system from entering an invalid or inconsistent state.

For example, during the exam generation phase, the system validates that every requested topic contains a sufficient number of questions to satisfy the randomization requirements. Similarly, the session state machine prevents an exam from transitioning to the Running state unless all physical booklets have been successfully compiled into PDF format. To implement these rules consistently, we utilize Pydantic schemas at the entry point of every service. These schemas act as the first line of defense, performing type coercion and structural validation before the data ever reaches the internal business logic. This ensures that the core services can operate on clean data, significantly reducing the potential for runtime errors and logical inconsistencies.

4.4.3 ORCHESTRATION OF RESOURCE-INTENSIVE OPERATIONS

The EduPro platform handles several operations that are computationally expensive or involve high-latency interactions with external services. To maintain a responsive user experience, the backend utilizes an asynchronous execution model that decouples these

long-running tasks from the standard request-response cycle.

When a professor initiates a batch generation of hundreds of unique exam booklets, the system does not block the user interface. Instead, the backend validates the request, stages the initial configuration, and delegates the heavy lifting to a background task queue. This includes specifically the LaTeX compilation of each booklet. A similar pattern is applied to the notification service, where the sequential dispatch of individualized emails to hundreds of students is performed asynchronously. By leveraging the asynchronous features of FastAPI and SQLAlchemy, the system can handle multiple concurrent generation requests without degrading the performance of other administrative tasks, allowing the platform to scale effectively during peak assessment periods.

4.5 OPTICAL MARK RECOGNITION

One of the core features of EduPro is the ability to automatically grade exams using [OMR](#). This section details the multi-step pipeline developed to process answer sheet images, from initial [QR](#) code detection to the final calculation of grades using deep learning classification.

4.5.1 OVERVIEW

The purpose of the [OMR](#) pipeline is to take a photograph of a completed answer sheet, locate the answer grid, extract each individual cell, classify it as *empty*, *erased*, or *selected* using a Convolutional Neural Network ([CNN](#)), and compute the student's grade. The pipeline is designed to handle imperfect camera angles, sheet rotation, and varying lighting conditions.

4.5.2 QR CODE DETECTION

The first step is to locate the [QR](#) code printed on the answer sheet. The [QR](#) code encodes the exam identifier (`Exam.id`) and serves both as a lookup key and as a geometric anchor for the grid localisation step. Detection is performed by the `get_robust_qr_bbox` function, which applies OpenCV's `QRCodeDetector` to five progressively preprocessed versions of the image: the original colour image, a greyscale conversion, a simple binary threshold, an adaptive Gaussian threshold, and a blurred adaptive threshold. The function returns the bounding box of the first successful detection. This multi-pass strategy significantly improves robustness on low-quality photographs, as illustrated in [Figure 4.5](#).

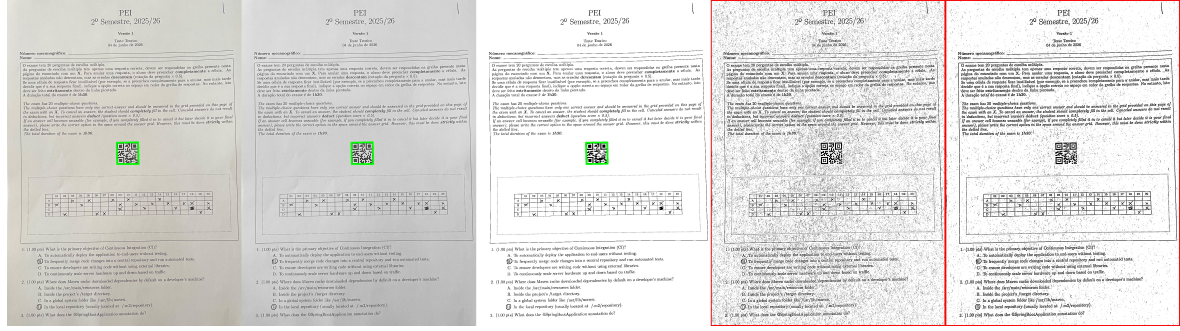


FIGURE 4.5 – Multi-pass QR Code Detection on an Answer Sheet

4.5.3 GRID LOCALISATION AND PERSPECTIVE CORRECTION

Once the QR code bounding box is known, the system locates the answer grid. The image is converted to greyscale, blurred with a 5×5 Gaussian kernel, and edge-detected with the Canny algorithm. A small 3×3 morphological closing operation is applied to repair minor breaks in the solid grid lines without connecting the widely-spaced dots of the outer decorative border, which would otherwise be mistaken for the grid.

Contours are extracted and sorted by area in descending order. Each candidate contour is evaluated against four geometric criteria: it must be located below the QR code, horizontally aligned with it, larger than the QR code area, and have an aspect ratio consistent with the expected grid dimensions (between 0.2 and 8.0). The first contour satisfying all criteria is selected. Its convex hull is computed and the four extreme corners (top-left, top-right, bottom-right, bottom-left) are identified algebraically from the sum and difference of coordinates.

Two perspective transforms are then computed from these four corners:

- A **tight crop** that warps the grid to a canonical rectangle with no padding. This image is used as input to the CNN.
- A **padded crop** that adds 15% horizontal and 60% vertical padding around the grid. This image is used as the canvas for drawing the colour-coded correction annotations, so that the context around the grid is visible in the output.

Both images are saved to disk alongside the original capture.

4.5.4 CELL EXTRACTION

The `extract_cells_math` function slices the tight-cropped grid into individual cells using pure arithmetic. The grid has $(\text{NUM_OPTIONS} + 1)$ rows and $(\text{num_questions} + 1)$ columns, where the first row and first column are header labels. The number of questions is read dynamically from the database (`Exam.answer_key`), so the pipeline adapts automatically to exams of different lengths without any hardcoded constants. The logical partitioning of the grid into individual classification units is shown in Figure 4.6.

Each cell is cropped with a 15% margin on each side to exclude grid lines, converted to greyscale, and resized to 32×32 pixels. The resulting array of cell images is stacked into a

	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A			X		X			X		X				X						
B	X			X					X		X		X		X		X	X		X
C									X			X				X				X
D		X				X	X												X	
	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A			X		X			X		X				X						
B	X			X					X		X		X		X		X	X		X
C									X			X				X				X
D		X				X	X												X	

FIGURE 4.6 – Grid Cell Extraction and Coordinate Mapping

NumPy array of shape $(\text{num_cells}, 32, 32, 1)$ and normalised to the $[0, 1]$ range before being passed to the [CNN](#).

4.5.5 EVOLUTION OF THE **omr!** (**omr!**) STRATEGY

The current [OMR](#) implementation is the result of an iterative development process aimed at overcoming the limitations of traditional image processing techniques. In the early stages of the project, the system utilized a traditional [OMR](#) logic based on pixel density analysis. In this initial implementation, the classification of each cell was determined by calculating the ratio of dark pixels within its boundaries. A primary threshold was established to identify selected options, while a significantly higher secondary threshold was used to categorize cells as erased, based on the assumption that an erasure would naturally leave more graphite or ink on the paper.

While this threshold-based method was computationally efficient, it proved to be highly susceptible to environmental and geometric variations. The most critical drawback was the occurrence of compounding errors during the grid extraction phase. Small misalignments or minor perspective distortions in the initial columns of the answer sheet would propagate across the table, resulting in increasing offsets for the latter columns. These offsets caused the extraction logic to capture parts of the grid lines or adjacent cells, leading to incorrect pixel density readings and frequent misclassifications.

The transition to a [CNN](#) provided a more robust solution to these challenges. By moving away from simple pixel counts to a model capable of hierarchical feature extraction, the system gained the ability to distinguish between intentional marks, erasures, and noise, even when cells were not perfectly centered. This change resulted in a substantial improvement in classification accuracy and overall system performance, enabling the platform to handle a wider variety of real-world photography conditions without requiring manual recalibration of thresholds.

4.5.6 CNN BASED CELL CLASSIFICATION

Cell classification is performed by a pre-trained [CNN](#) loaded at application startup. The model was trained to distinguish three classes: **empty** (the student did not mark this option),

erased (the student marked and then erased this option), and **selected** (the student’s final answer). Loading the model once at startup avoids the overhead of reloading it on every request.

All cells for a given exam are classified in a single batched `model.predict` call, which is efficient even for exams with many questions. The predicted class index for each cell is obtained via `np.argmax` on the softmax output.

4.5.7 SCORING AND ANNOTATION

After classification, the `evaluate_exam` function iterates over each question and collects the cells classified as **selected** or **erased**. For each question, the answer is compared against the stored answer key. The grade contribution of each question is computed as:

$$s_q = \frac{w_q}{\sum_i w_i} \times 20 \times \left(c_q - \frac{f}{100} \times e_q \right)$$

where w_q is the relative weight of question q , c_q is 1 if the selected answer is correct and 0 otherwise, e_q is the number of incorrect selected answers, and f is the fractional penalty percentage stored in `ExamConfig.fraction`. The final grade is the sum of all s_q , clamped to a minimum of 0.

The padded image is annotated with colour-coded rectangles: green for a correctly selected cell, red for an incorrectly selected cell, and blue for an erased cell. The annotated image is saved to disk and its path stored in `Exam.correction_path`. The raw grade, the per-question answer dictionary, and the capture path are written back to the `Exam` record. After each evaluation, the warning service recalculates all warnings for the `ExamConfig` to reflect the updated state.

4.6 WEB APPLICATION

The EduPro frontend was built with the goal of making the complex administrative workflows described in earlier chapters feel natural and self-explanatory for every role in the system. Because the platform serves users with very different responsibilities, from a manager configuring institutional structure to a vigilant scanning exam papers on site, the interface had to remain consistent and intuitive across radically different contexts. The design therefore prioritised clear feedback, progressive disclosure of complexity, and a coherent transition between desktop-heavy administrative tasks and mobile-first interactions.

The platform entry point is the landing page, which acts as the public gateway before any authentication takes place. The page is deliberately minimal, presenting the platform identity and two clear entry points: creating a new account or signing in to an existing one. The interface supports both light and dark display modes, adapting to the user’s operating system preference, as shown in Figure 4.7.

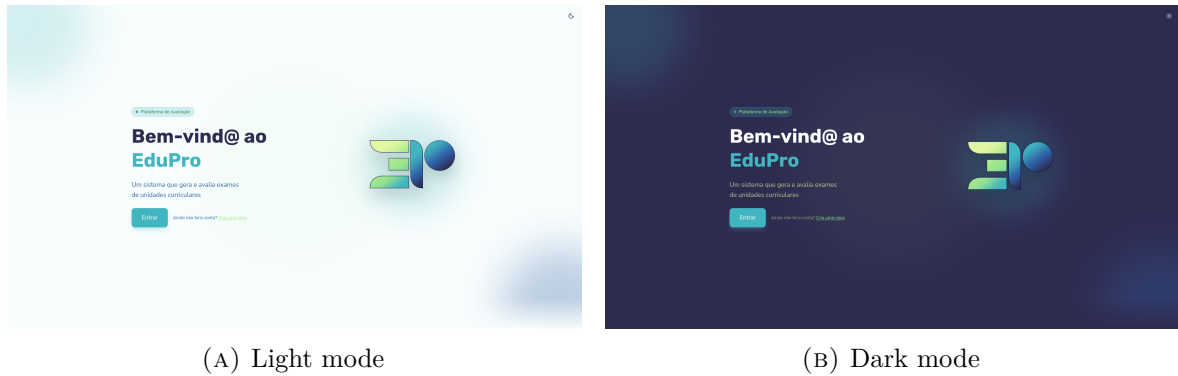


FIGURE 4.7 – EduPro landing page in light and dark mode.

When a user chooses to create a new account, the platform redirects to a Keycloak-hosted registration form. The form performs validation before submission and highlights any missing or malformed fields directly beneath the respective input, as illustrated in Figure 4.8. This immediate feedback prevents the user from submitting an incomplete request and reduces friction during onboarding.

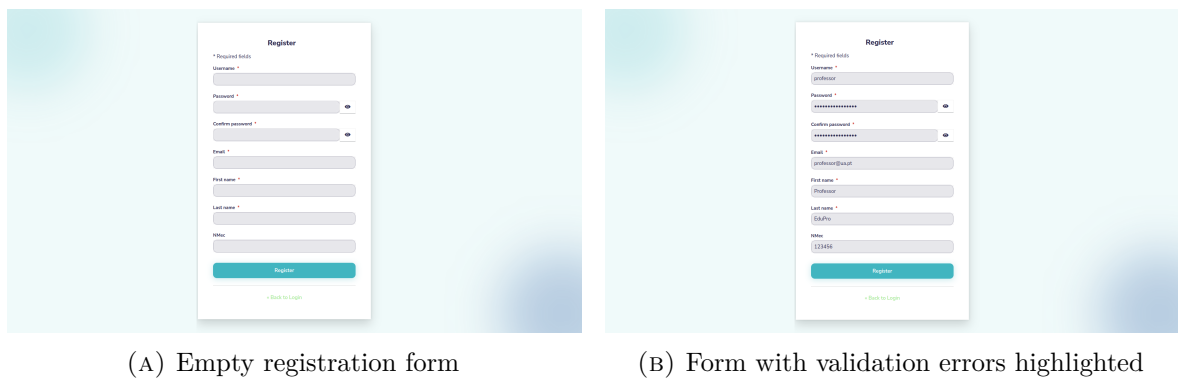


FIGURE 4.8 – Keycloak registration form showing the empty state and inline validation feedback.

For users who already have an account, the login form follows the same Keycloak-hosted pattern. When incorrect credentials are submitted, the form surfaces a clear error message without disclosing whether the username or the password was at fault, in keeping with standard security practice. Figure 4.9 shows the login form alongside the error state.

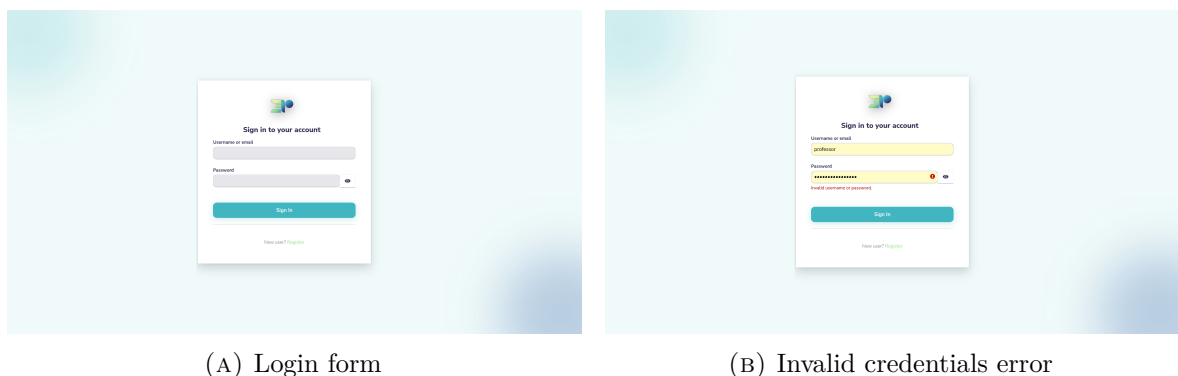


FIGURE 4.9 – Keycloak login form and error feedback.

4.6.1 CURRICULAR UNIT MANAGEMENT

Once authenticated, every user lands on the curricular units page, which serves as the central hub of the application. The experience at this point differs according to role. A manager has full visibility over every curricular unit in the system and has access to actions for creating, editing, and deleting them, while a search bar at the top of the list allows rapid filtering across all available units. Before any units exist, the page presents an empty state with a call to action, as shown in Figure 4.10.



FIGURE 4.10 – Curricular units page before any unit is created.

Once units have been created they appear as cards in the list, each showing the unit name and code alongside the available management actions, as shown in Figure 4.11.



FIGURE 4.11 – Curricular unit list with existing units.

A new curricular unit is created through a dedicated button that opens a modal dialog

over the current page, keeping the user in context. The form requests the unit name and code, and submission is only enabled once all required fields are populated. After confirmation, the new unit appears immediately in the list, as shown in Figure 4.12.

Figure 4.12 consists of two side-by-side screenshots of a web application interface. Both screenshots show a modal dialog titled 'Nova Unidade Curricular' (New Curricular Unit) overlaid on a background of 'Unidades Curriculares' (Curricular Units). The dialog has a 'Nome' (Name) field and a 'Código' (Code) field. Below these fields are two sections: 'Regente' (Coordinator) and 'Outros Docentes' (Other Teachers). Each section has a search bar and a list of available teachers. In screenshot (A), the form is empty. In screenshot (B), the form is populated with data, including the name 'Projeto em Engenharia Informática', the code '000000000', and the coordinator 'Professor Edipo antoniobasilio'. The 'Outros Docentes' section also shows a list of teachers.

(A) Empty creation form

(B) Form ready for submission

FIGURE 4.12 – Modal dialog for creating a new curricular unit.

Unit editing follows the same modal pattern, presenting a visually identical form pre-populated with the existing values. This consistency ensures that the manager does not need to learn a separate interface for modification, as illustrated in Figure 4.13.

Figure 4.13 shows a modal dialog titled 'Editar Projeto em Engenharia Informática' (Edit Project in Computer Engineering) overlaid on the same background as Figure 4.12. The dialog is pre-populated with the same data as in Figure 4.12(B), including the name 'Projeto em Engenharia Informática', the code '000000000', and the coordinator 'Professor Edipo antoniobasilio'. The 'Outros Docentes' section also shows a list of teachers. The dialog has a 'Cancelar' (Cancel) button and a 'Guardar' (Save) button.

FIGURE 4.13 – Modal dialog for editing an existing curricular unit.

Deletion is a two-step interaction designed to prevent accidental data loss. The manager selects the delete action on a specific unit, which triggers a confirmation dialog, as shown in Figure 4.14.



FIGURE 4.14 – Delete action selected on a curricular unit.

Only after explicit confirmation in the dialog is the unit permanently removed, as shown in Figure 4.15.

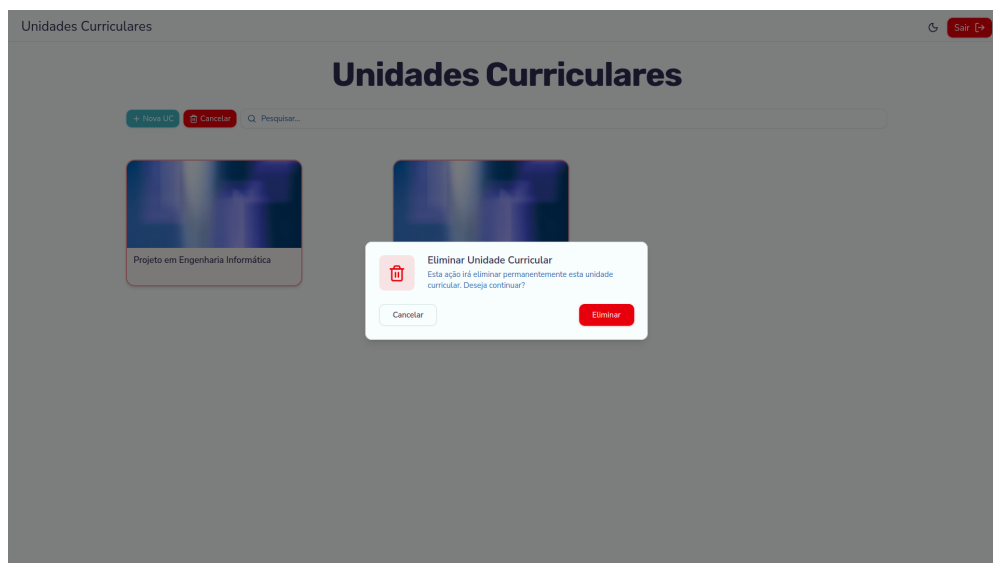
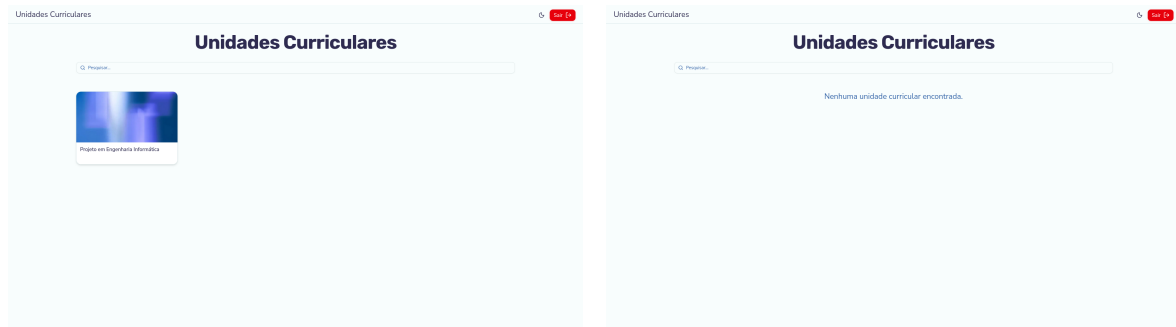


FIGURE 4.15 – Deletion confirmation dialog.

4.6.2 SUBJECT ADMINISTRATION

A user authenticated as a regent or professor sees a narrower view of the same curricular units page. Only the units to which they have been explicitly assigned appear in the list, and the interface exposes only the actions relevant to their role, with the creation and deletion controls reserved for managers. A professor who has not yet been assigned to any unit is presented with an empty state that communicates this clearly, as shown in Figure 4.16.



(A) View with assigned units

(B) View with no assigned units

FIGURE 4.16 – Curricular units page for non-manager roles.

Upon selecting a unit, the subject detail view opens, organised into tabs covering the three core areas of administration: the teaching team, the question bank, and the exam sessions. A regent who has been granted regent-level permissions on a unit gains access to all three areas and their associated editing capabilities, as shown in Figure 4.17.



FIGURE 4.17 – Subject detail view as seen by a regent.

The professors tab allows the regent to manage who is assigned to the unit at the professor level. Editing the list is performed through a multi-select input that searches across all registered users in the identity provider. The regent types to search and selects the desired professors from the dropdown, as shown in Figure 4.18.

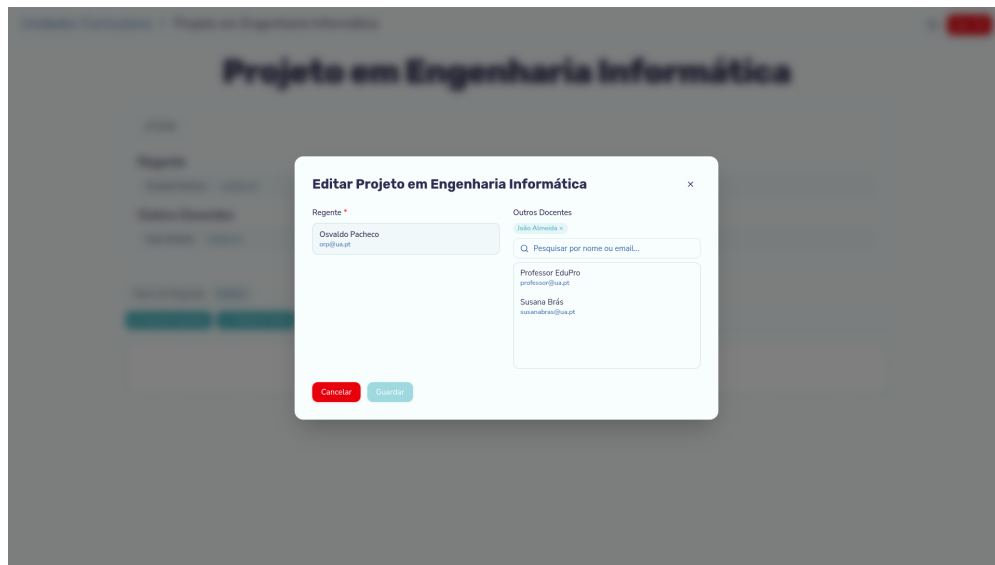


FIGURE 4.18 – Professor assignment form before any selection.

Once the selection is confirmed, the updated list is saved and reflected immediately in the view, as shown in Figure 4.19.



FIGURE 4.19 – Professor list updated after confirmation.

The question bank tab is where the regent builds and maintains the pool of questions drawn from during exam generation. The content is organised by topic, and questions can be populated either through bulk import from an external system or by creating topics and questions individually.

For situations where a large set of questions already exists in an external system, the platform supports bulk import from [XML](#) files. A help tooltip on the import button, shown in Figure 4.20, describes the expected file structure: a root `quiz` element containing `question` entries, each with a `name`, a `questiontext`, and a set of `answer` elements where the correct option is identified by a non-zero grade attribute.



FIGURE 4.20 – Import hint tooltip describing the expected **XML** format.

After selecting a file, a preview of the parsed topics and questions is presented for review before any data is committed, as shown in Figure 4.21.

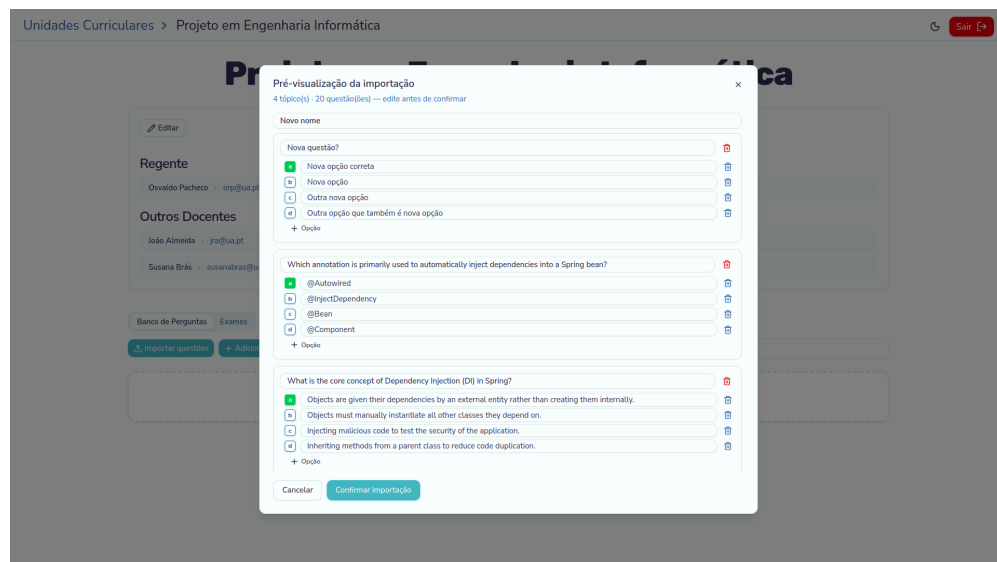


FIGURE 4.21 – Preview of parsed questions before import confirmation.

Upon confirmation, all topics and questions are added to the bank in a single operation, as shown in Figure 4.22.



FIGURE 4.22 – Question bank after bulk import.

Alternatively, the bank can be built manually. Topics act as thematic categories for the questions and determine how the exam generator distributes draws across the subject matter. A new topic is created through a modal that requires only a name, as shown in Figure 4.23.

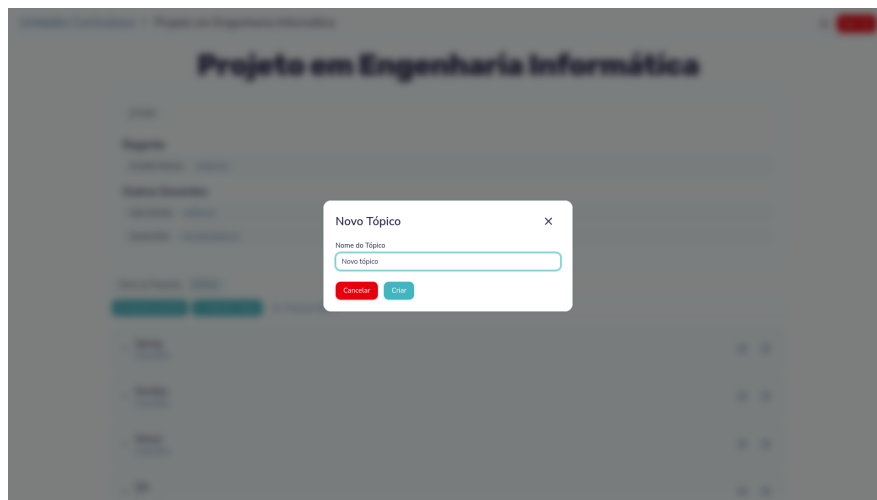


FIGURE 4.23 – Topic creation modal.

After confirmation the topic appears as a collapsible section in the bank, ready to receive questions, as shown in Figure 4.24.

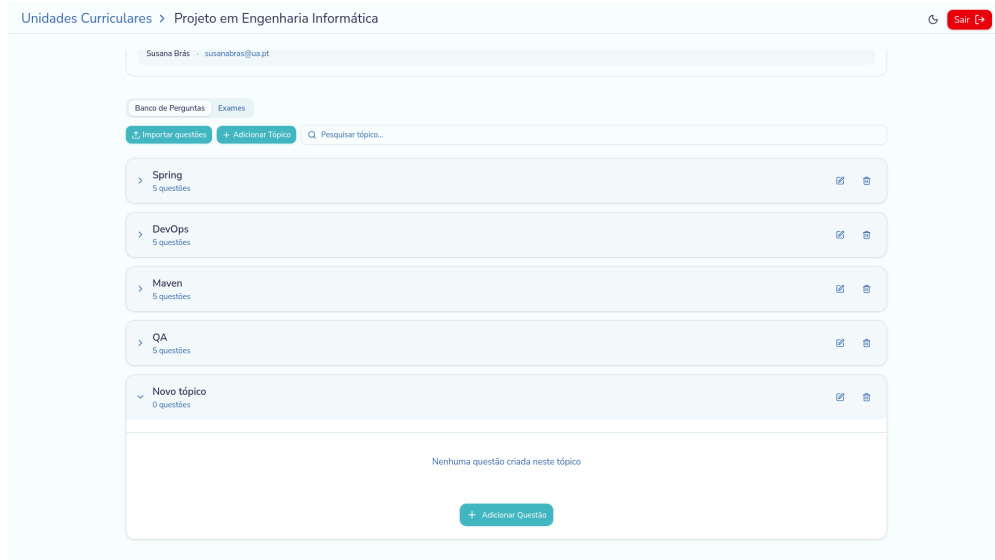


FIGURE 4.24 – Newly created topic expanded in the question bank.

Within a topic, questions are added one at a time through a modal that requires a statement and at least two answer options, with exactly one marked as correct. The form enforces these constraints inline before allowing submission, as shown in Figure 4.25.

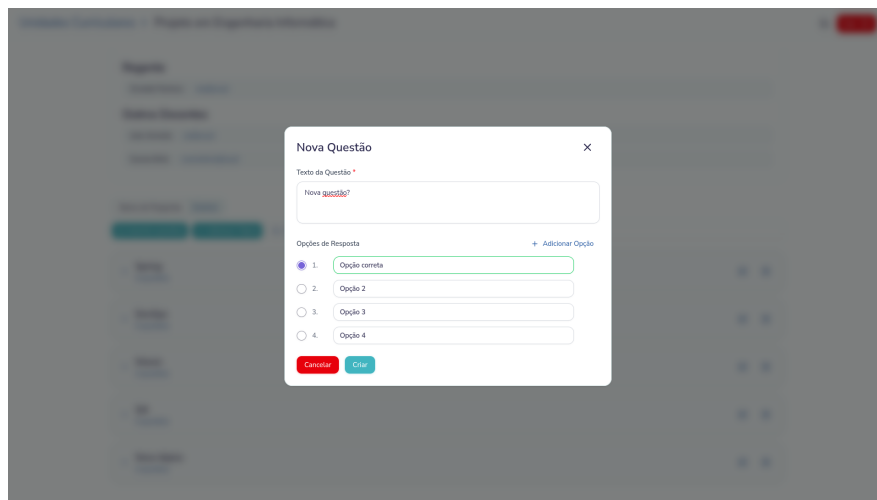


FIGURE 4.25 – Question creation modal filled and ready for submission.

If the constraints are not met, the form highlights the incomplete fields before allowing the regent to proceed, as shown in Figure 4.26.

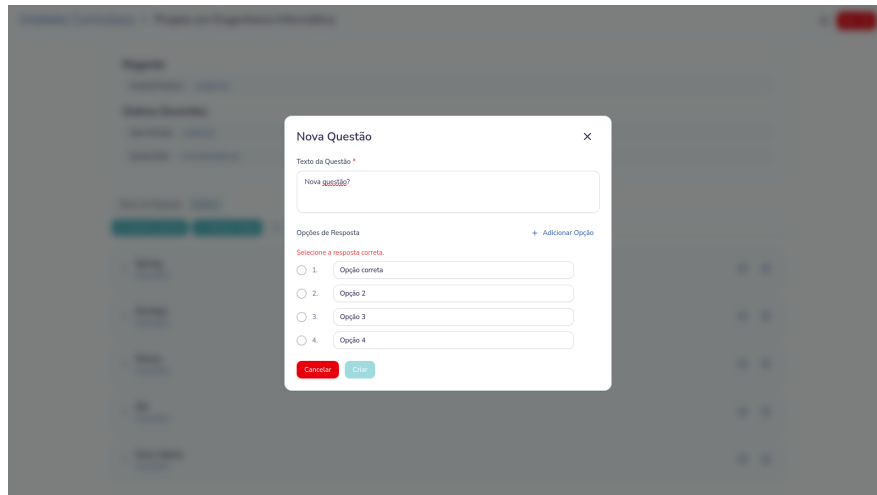


FIGURE 4.26 – Inline validation feedback when question constraints are not met.

After saving, the question appears inside its topic in the bank, as shown in Figure 4.27.

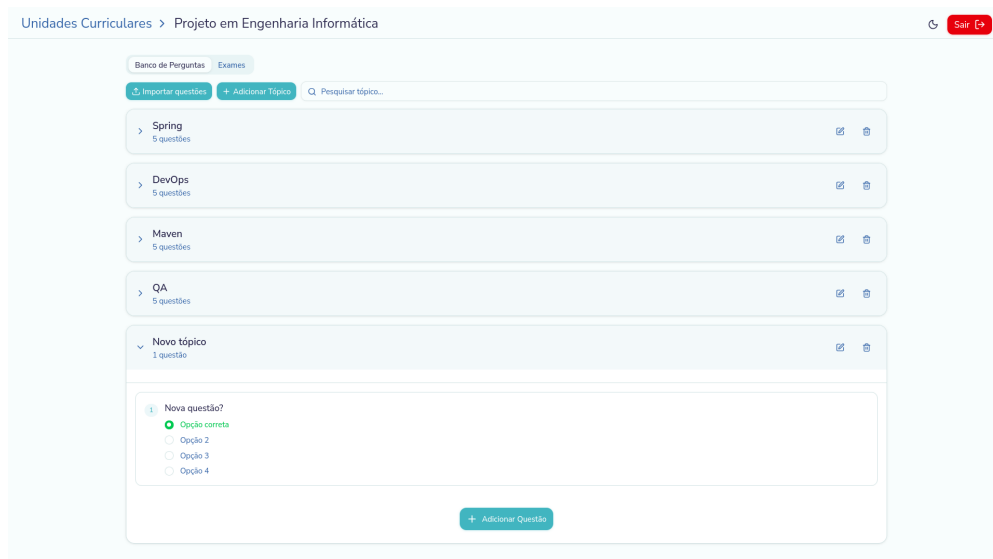


FIGURE 4.27 – Question added to the bank after creation.

A topic name can be updated at any time through the edit button next to the topic row. This opens a modal pre-populated with the current name, as shown in Figure 4.28.

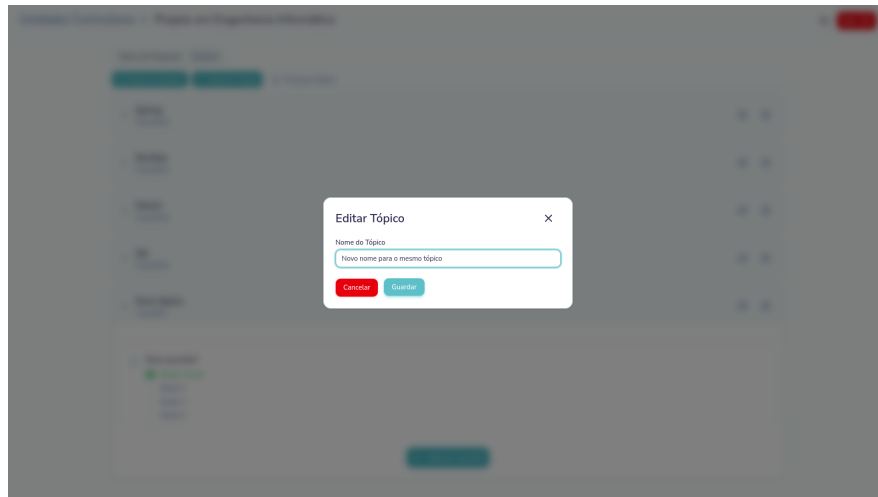


FIGURE 4.28 – Topic name edit modal with updated name.

After saving, the topic reflects the new name in the bank, as shown in Figure 4.29.

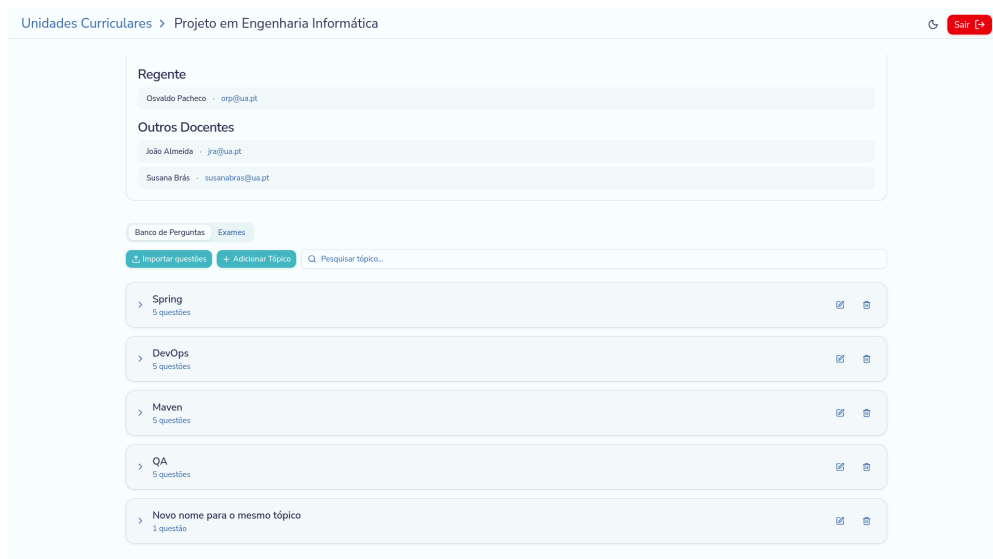


FIGURE 4.29 – Question bank showing the topic with its updated name.

After a question is saved it appears inside its topic and can be edited at any time through the same modal pre-populated with its current content, as shown in Figure 4.30.

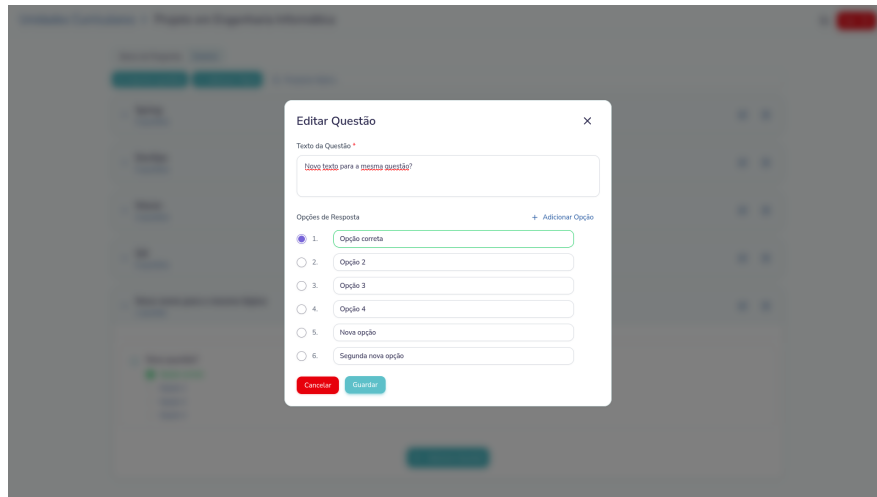


FIGURE 4.30 – Edit question modal with updated content.

After saving, the question reflects the changes in the bank, as shown in Figure 4.31.

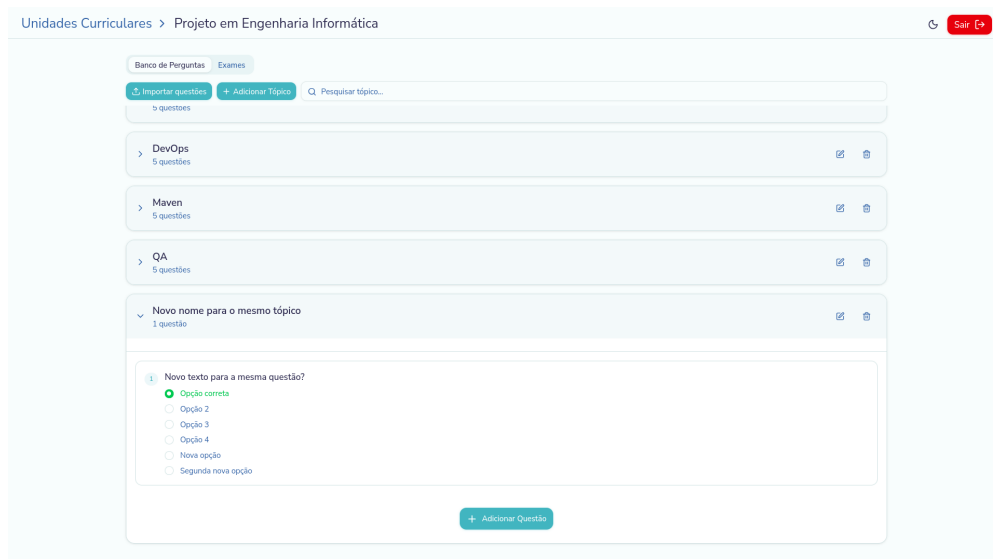
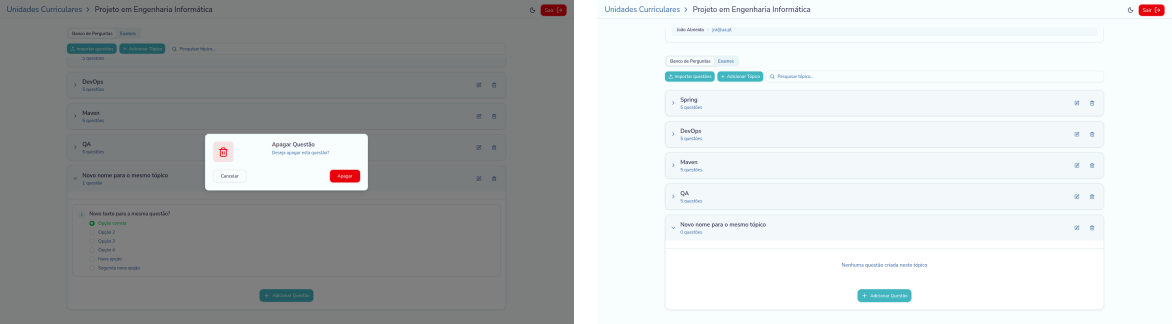


FIGURE 4.31 – Updated question in the bank after editing.

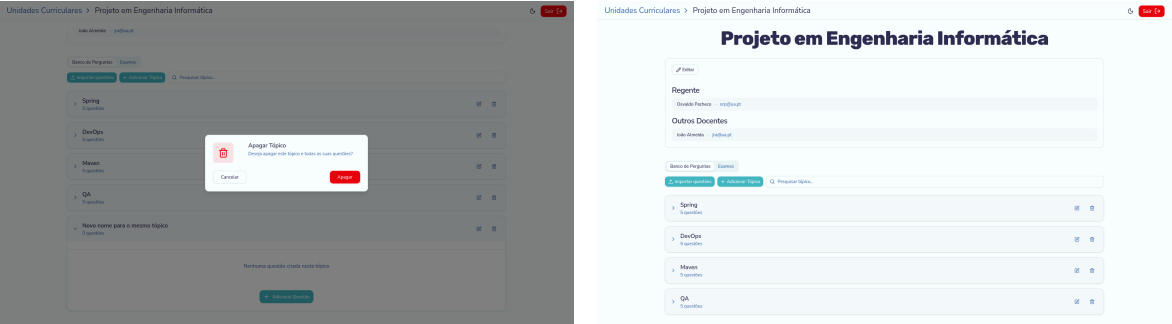
Question deletion follows the same two-step confirmation pattern used throughout the interface. The regent selects the delete action on the question, a confirmation dialog appears, and upon confirmation the question is removed from the topic, as shown in Figure 4.32.



(A) Delete confirmation dialog (B) Topic after question removal

FIGURE 4.32 – Two-step deletion of a question from the bank.

Topics can equally be removed when no longer needed, following the same confirmation pattern. Figure 4.33 shows the deletion prompt and the bank after the topic has been removed.



(A) Topic deletion confirmation (B) Bank after topic removal

FIGURE 4.33 – Two-step deletion of a topic from the question bank.

The exams tab is the starting point for initiating a new assessment session. A regent who has not yet created any exam for the unit is shown an empty state with a prominent call to action, as seen in Figure 4.34. The creation button opens a multi-step modal that guides the regent through every parameter required to define an exam session.



FIGURE 4.34 – Exams tab before any session has been created.

The first step of the modal asks the regent to select the topics from which questions will be drawn. This selection defines the thematic scope of the exam and determines which parts of the question bank are eligible for generation. A searchable list of all available topics in the unit makes it straightforward to find and include the relevant ones, as shown in Figure 4.35.

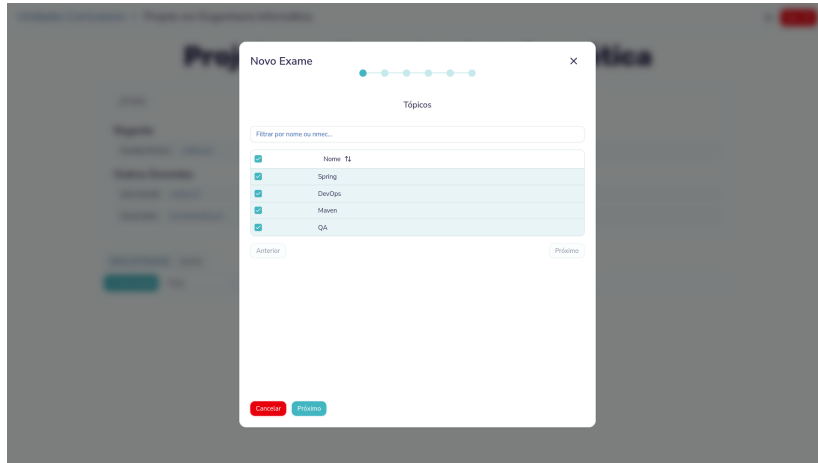


FIGURE 4.35 – Exam creation step 1: topic selection.

The second step determines how many questions are drawn from each selected topic. The form validates that the requested count does not exceed the number of questions available in each topic, preventing configurations that cannot be fulfilled at generation time. This value directly controls the total length of the exam paper, as shown in Figure 4.36.

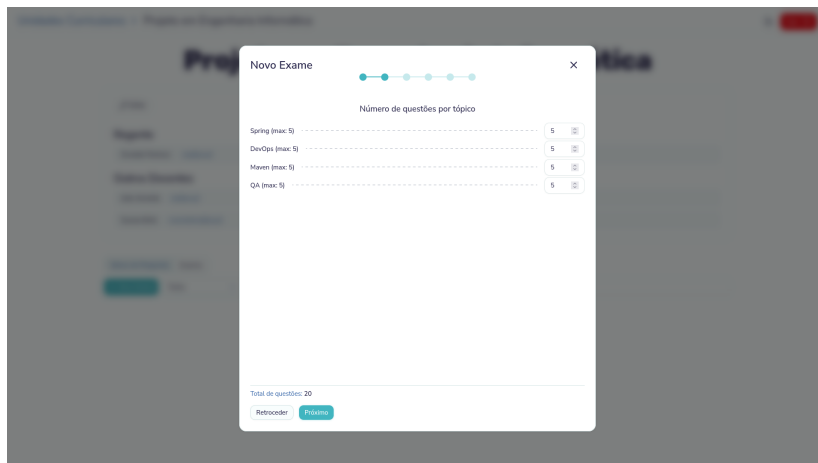


FIGURE 4.36 – Exam creation step 2: number of questions per topic.

The third step assigns a weight to each topic and a global penalty fraction for wrong answers. The weights determine the relative contribution of each topic to the final grade, while the penalty is applied by the grading formula to discourage random guessing. Figure 4.37 shows this step with integer weights configured.

Tópico	Modo	Pontuação máxima	Porcentagem
Spring		0.4 voto	1
DevOps		0.8 voto	2
Maven		1.2 voto	3
QA		1.6 voto	4

Desconto (%) 35

Retornar Avançar

FIGURE 4.37 – Exam creation step 3: topic weights and wrong-answer penalty.

The fourth step configures the session metadata, namely the exam name and date. These fields identify the session throughout the platform and appear on the generated exam papers and in the exported grade reports, as shown in Figure 4.38.

Configurações finais

Título do exame * Exame Final

Data do exame * 13/06/2026

Semestre * 2º Semestre

Ano letivo * 2025/26

Retornar Avançar

FIGURE 4.38 – Exam creation step 4: session name and date.

The fifth step handles enrollment. The student list is imported via a [CSV](#) file containing three columns: the student number, the first name, and the institutional email address. A help tooltip on the import button makes this format explicit, as shown in Figure 4.39.

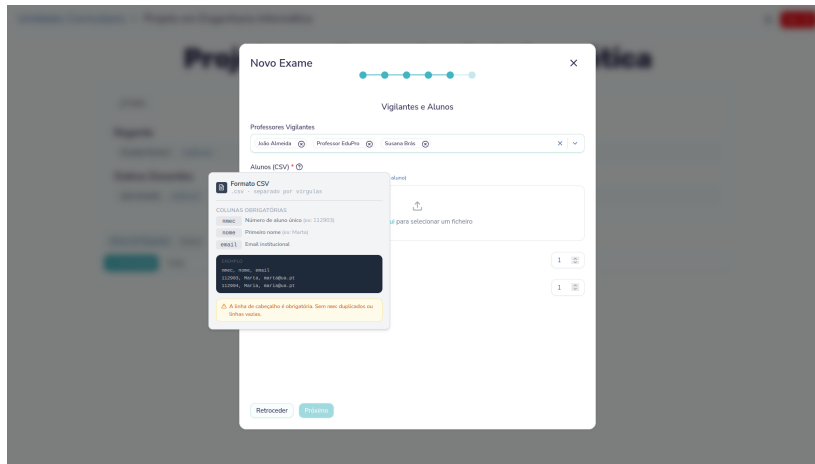


FIGURE 4.39 – Exam creation step 5: CSV format tooltip.

The regent also selects the vigilants who will supervise the exam, granting them access to the mobile association interface during the live session. After file selection, the parsed students are previewed alongside the selected vigilants before confirmation, as shown in Figure 4.40.

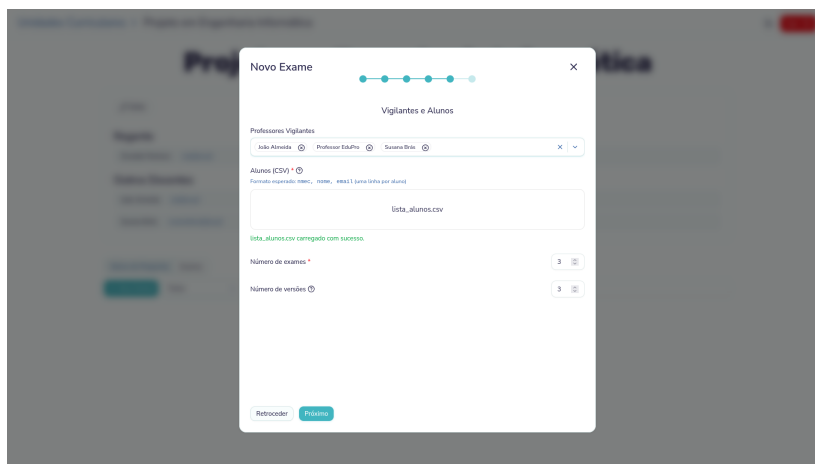


FIGURE 4.40 – Exam creation step 5: vigilants selected and students imported.

The final step presents a complete summary of all configured parameters across every preceding step. The regent can review everything before committing and navigate back to correct any detail, as shown in Figure 4.41.

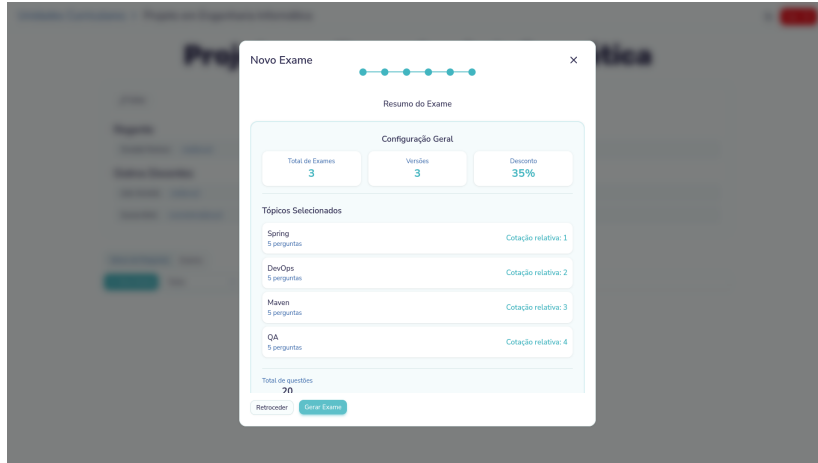


FIGURE 4.41 – Exam creation final step: full parameter summary before submission.

Upon submission, the backend generates the randomized exam papers asynchronously. The exam card initially appears in a processing state and transitions to concluded once generation is complete, as shown in Figure 4.42.

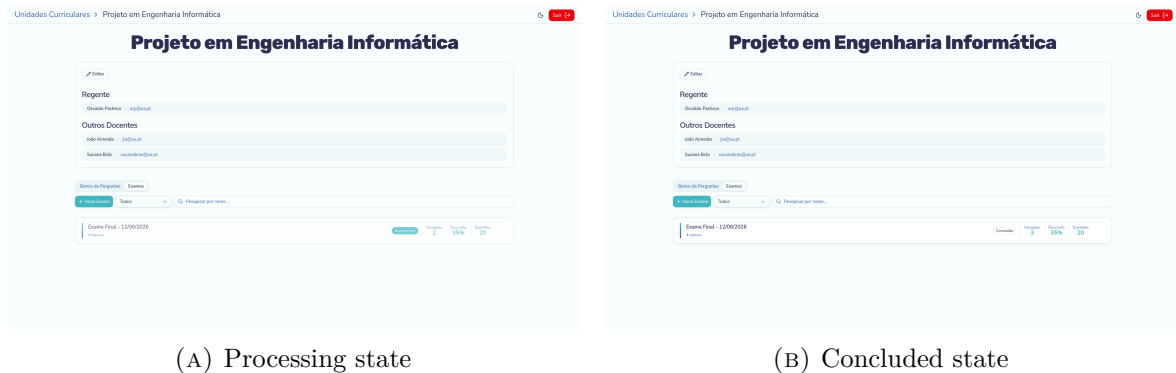


FIGURE 4.42 – Exam card transitioning from processing to concluded after generation.

With the exam generated, selecting it opens the session detail page, the operational centre for managing the entire assessment lifecycle. This page is built around an eight-step sequential stepper that mirrors the state machine described in Section 4.3. Each step expands to reveal the actions relevant to that phase, and steps that have not yet been reached remain locked, preventing out-of-order interaction, as shown in Figure 4.43.

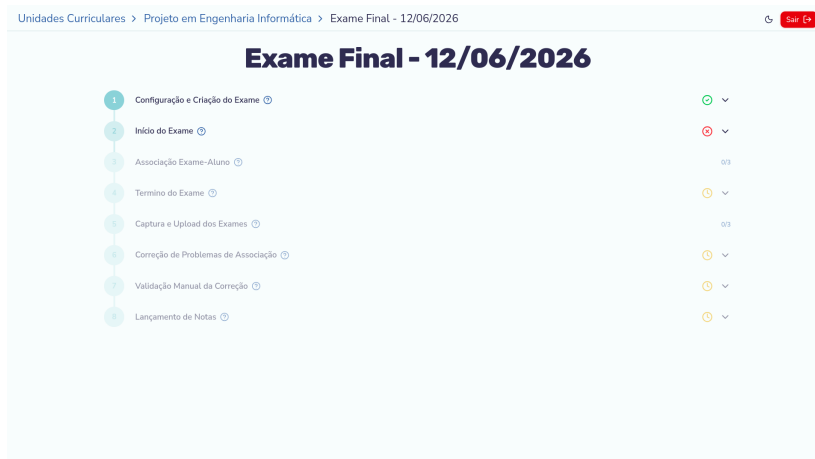


FIGURE 4.43 – Session detail page showing the eight-step stepper.

A key design aspect of the stepper is that certain steps are operable on desktop while others are exclusively mobile. Steps 3 and 5, student association and answer sheet capture respectively, have no expandable content on desktop because those physical actions can only be performed in the classroom using a smartphone. On desktop they appear as progress counters, showing in real time how many associations or captures have been completed out of the total. Steps 2 and 4 can be triggered from either device, while the remaining steps are desktop only.

Step 1 is always the starting point and displays the full configuration summary for review. It also provides the download button for the generated exam package, containing the randomized exam papers ready for printing and the corresponding answer keys, as shown in Figure 4.44.



FIGURE 4.44 – Step 1 expanded: configuration summary and exam package download.

Step 2 allows the regent to confirm the list of vigilants and start the exam session. Until this step is triggered, vigilants cannot access the mobile interface, as the session remains closed. Once started, the session opens and vigilants can begin associating students to exam papers. This action is available on both desktop and mobile.

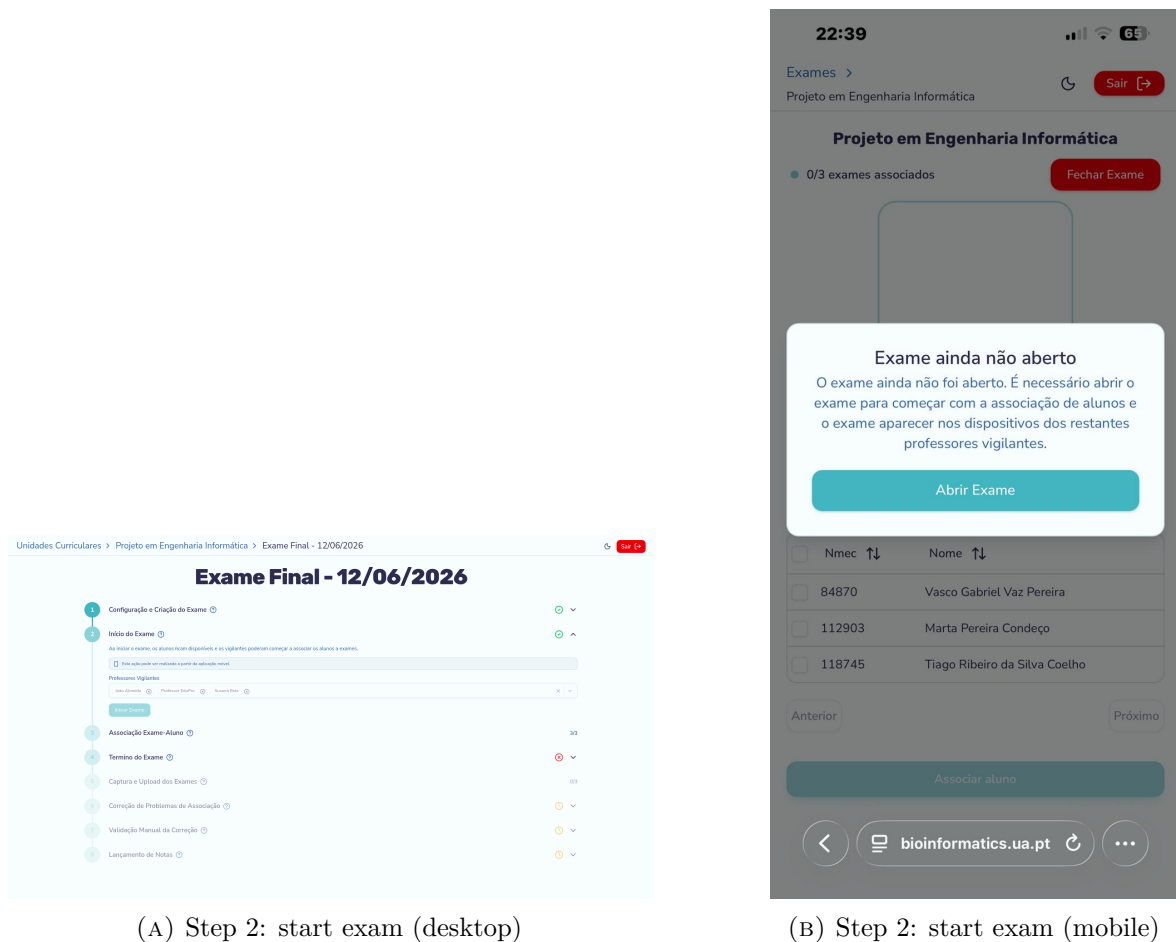
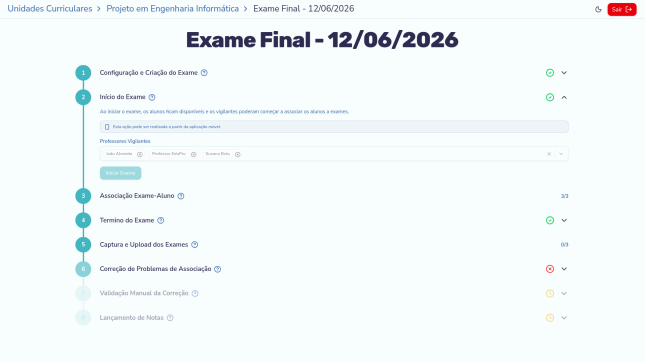
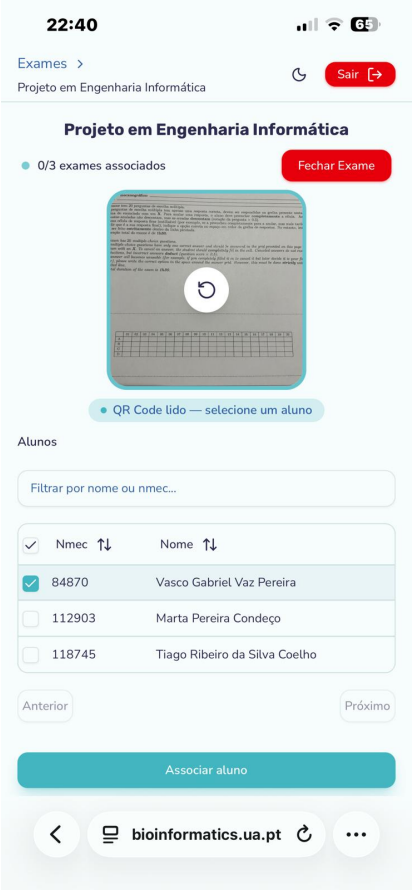


FIGURE 4.45 – Starting the exam session on desktop and mobile.

Step 3 is exclusively mobile. The association between a student and a physical exam paper is performed by scanning the QR code printed on the paper with the phone camera, making a desktop equivalent impossible. Once the exam is running, vigilants use their phones to scan each paper and link it to the corresponding student. The desktop stepper displays a live counter of completed associations while all action takes place on the mobile interface, as shown in Figure 4.46.



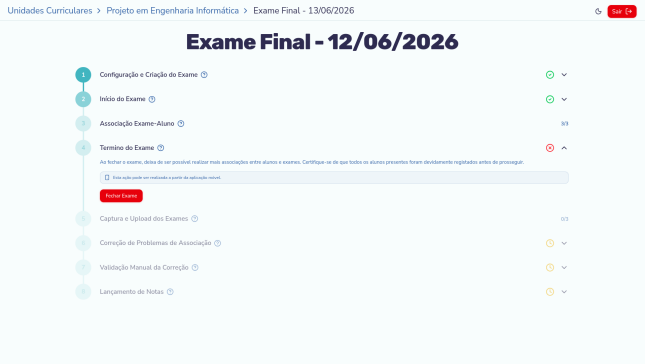
(A) Step 3: association progress counter (desktop)



(B) Step 3: QR scanner (mobile)

FIGURE 4.46 – Student-exam paper association on desktop and mobile.

Step 4 closes the exam session, preventing any further associations. This action can be triggered from both desktop and mobile. Figure 4.47 shows the step expanded on desktop and the equivalent action on mobile.



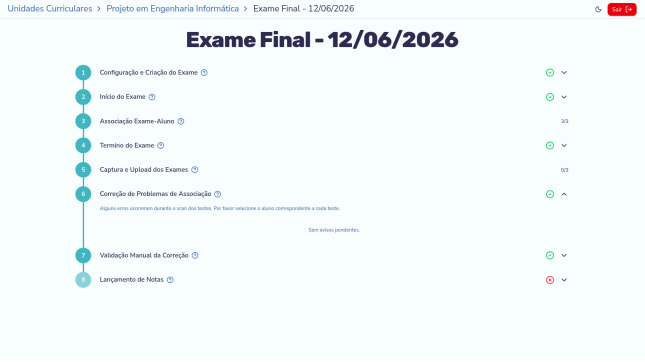
(A) Step 4: close exam (desktop)



(B) Step 4: close exam (mobile)

FIGURE 4.47 – Closing the exam session on desktop and mobile.

Step 5 is exclusively mobile. After the physical exam papers are collected, vigilants photograph each answer sheet and submit it through the mobile interface. The autograding engine processes each image in the background. As with Step 3, the desktop stepper shows a live upload counter while all action takes place on the phone, as shown in Figure 4.48.



(A) Step 5: upload progress counter (desktop)



(B) Step 5: camera capture interface (mobile)

FIGURE 4.48 – Answer sheet capture and upload on desktop and mobile.

Once all sheets have been submitted, Step 6 handles any anomalies detected during processing. The regent is presented with a list of conflicting exam papers, each showing the candidate students associated with it, and must manually select the correct student for each conflict before the session can advance. Figure 4.49 shows the conflict resolution interface.

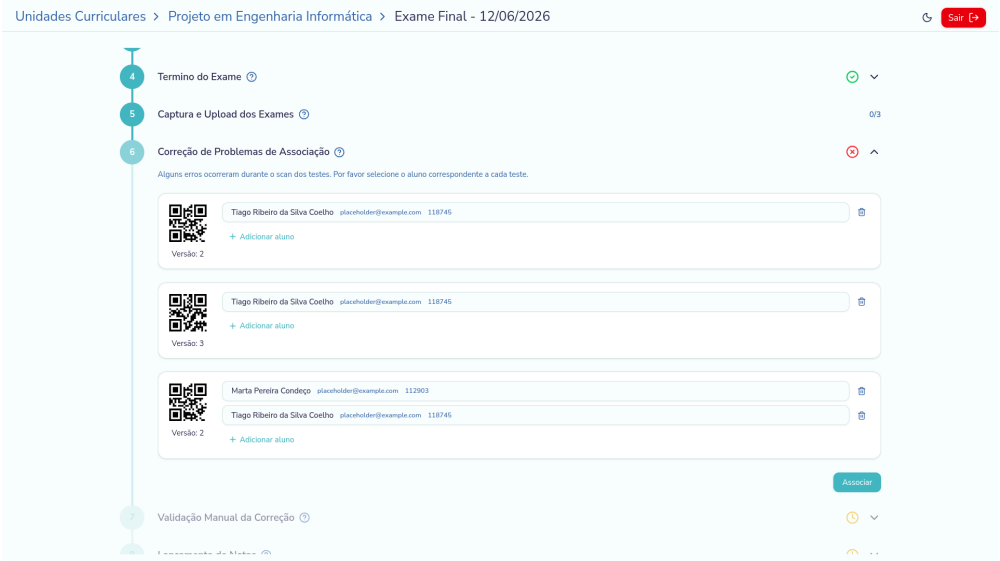
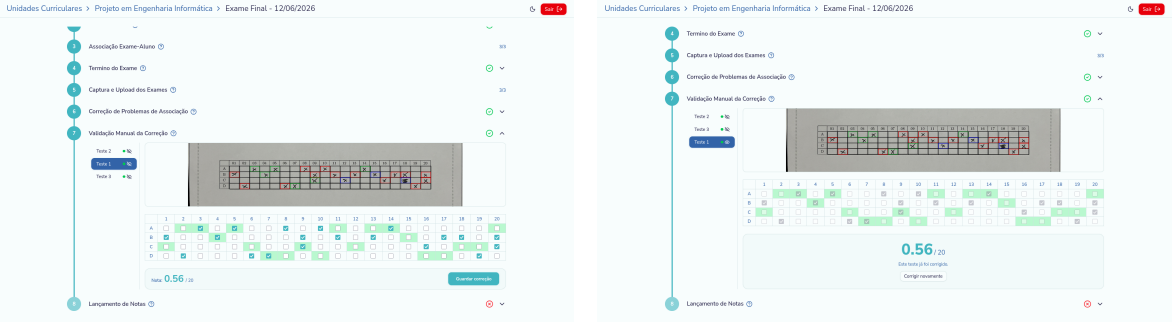


FIGURE 4.49 – Step 6: conflict resolution interface for association anomalies.

Once all conflicts have been resolved, Step 7 presents each automatically graded exam for manual review. For each exam the regent sees the autograding result and a checkbox to confirm it. If the autograding classification is correct, the regent checks the box to validate it. If a mark was misread, the regent can edit the answer before confirming. Figure 4.50 shows an exam before and after the correction is applied.



(A) Exam pending review (B) Exam validated

FIGURE 4.50 – Per-exam manual review and validation in Step 7.

Once every exam has been validated, Step 8 becomes available. Before releasing the grades, the regent composes the notification email using four content toggles and a free-text field. Each toggle controls an optional section appended to every student email: the photo of the scanned answer sheet, the question weights and penalty table, a colour-coded answer grid showing correct and incorrect selections, and a cumulative score table breaking down each question’s contribution to the final grade. The free-text field allows a personalised message at the top of every email. A live preview updates as options are toggled, as shown in Figure 4.51.

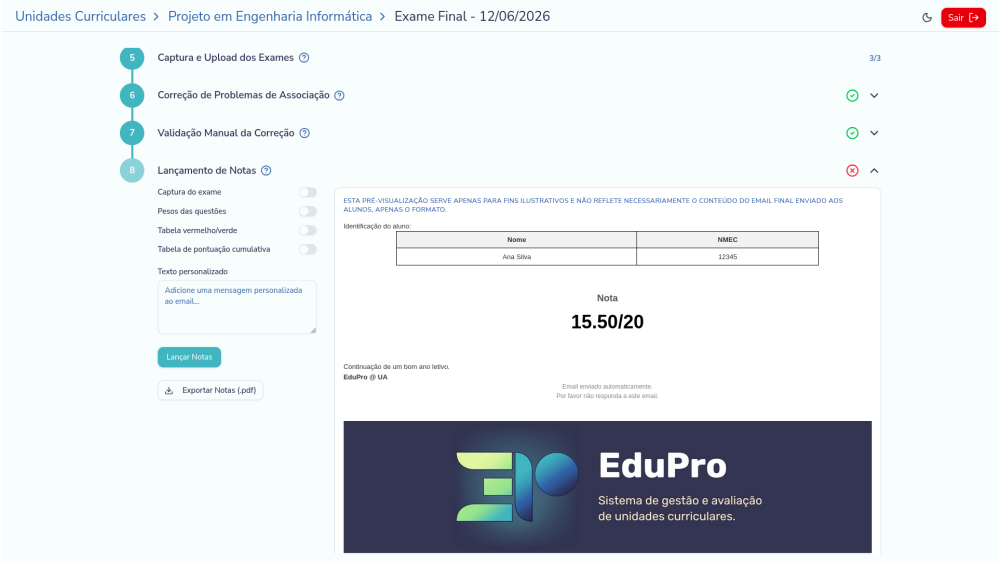


FIGURE 4.51 – Step 8: email composition with content toggles and live preview.

The minimal email sent to the student always includes identification and the final grade. Figure 4.52 shows this base view.

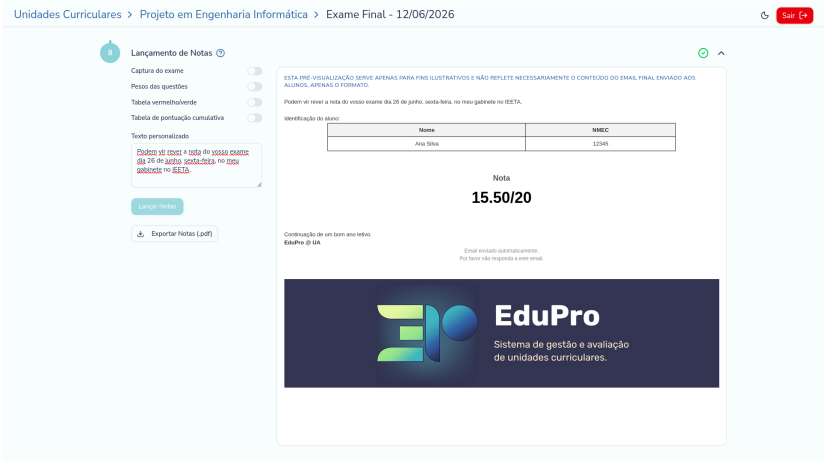


FIGURE 4.52 – Student email: grade only.

Enabling the answer grid toggle adds a colour-coded table where green cells indicate correct options and red cells indicate incorrect selections made by the student, giving immediate feedback on each question, as shown in Figure 4.53.

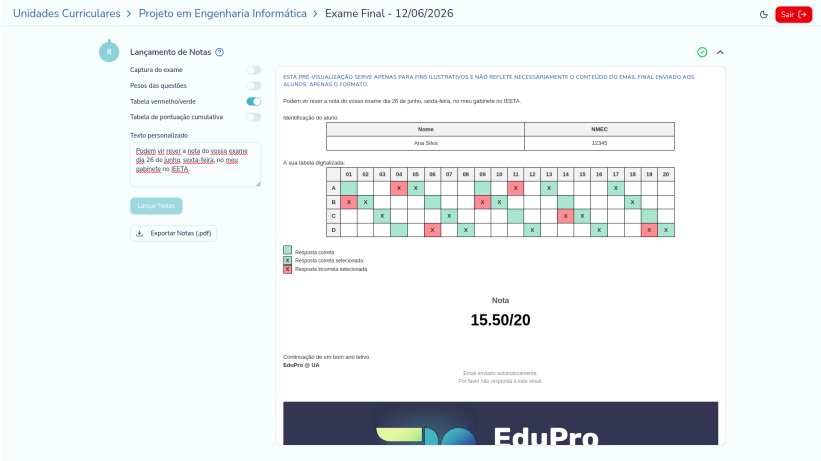


FIGURE 4.53 – Student email: grade and colour-coded answer grid.

The question weights toggle appends a table listing the point value and penalty of each question, allowing the student to understand how their score was computed, as shown in Figure 4.54.

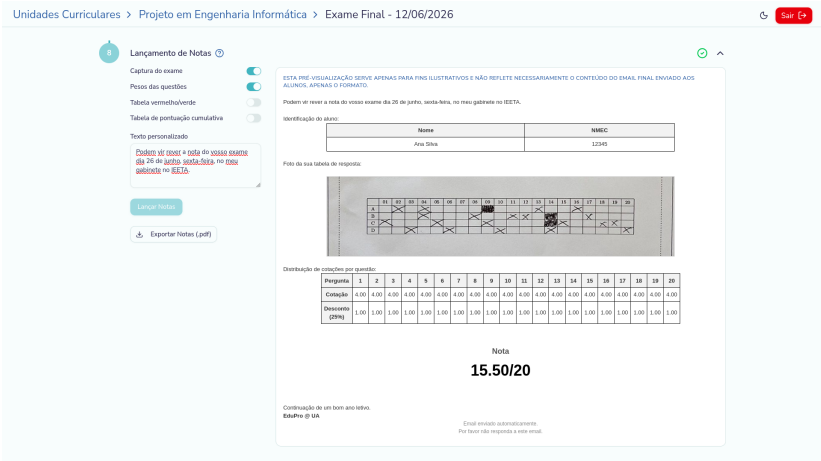


FIGURE 4.54 – Student email: answer grid and question weights table.

Finally, the cumulative score toggle adds a per-question breakdown showing correct and incorrect counts, the score contribution of each answer, and the running total at every step. Figure 4.55 shows the email with all options enabled.

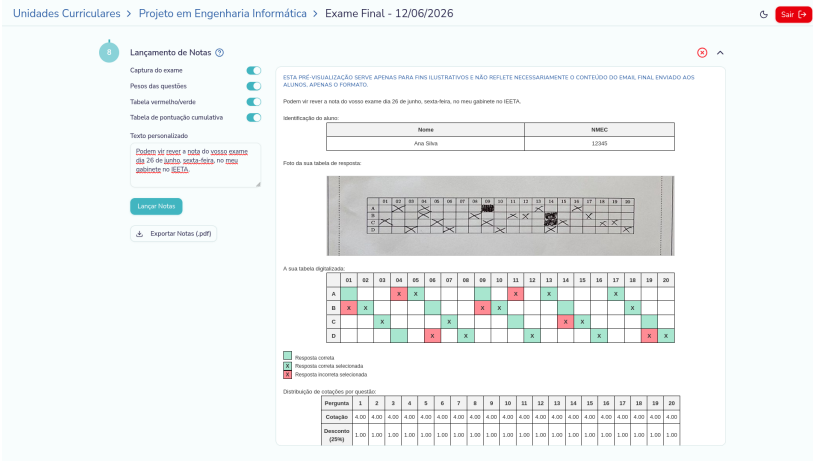


FIGURE 4.55 – Student email: all options enabled including cumulative score table.

Once the configuration is complete, the regent triggers the grade release, which dispatches the email to all enrolled students. The grades can also be exported as a PDF file for institutional record-keeping, as shown in Figure 4.56.

Relatório de Notas

Projeto em Engenharia Informática

2º Semestre, 2025/26

Exame Final

12 de junho de 2026

NMEC	Nome	Nota
84870	Vasco Gabriel Vaz Pereira	0.56
112903	Marta Pereira Condeço	16.60
118745	Tiago Ribeiro da Silva Coelho	9.76

FIGURE 4.56 – Exported grades at the end of the assessment lifecycle.

4.6.3 FRONTEND TECHNOLOGIES AND ARCHITECTURE

The technical realization of the EduPro frontend is built on a modern, type-safe stack that prioritizes performance and maintainability. The application is developed as a Single-Page Application (SPA) using React and TypeScript, leveraging a file-based routing system provided by TanStack Router that ensures full type safety for all navigation and Uniform Resource Locator (URL) parameters. This architectural choice eliminates a common class of runtime errors and provides a highly predictable development environment.

Server-side synchronization and state management are handled by TanStack Query, which orchestrates the complex interaction with the backend API. This layer is particularly important for the real-time aspects of the platform, such as monitoring the progress of student associations and autograding processing. The visual layer is constructed using Tailwind CSS for utility-first styling, combined with a library of accessible UI primitives from Radix UI. These components ensure that the application remains accessible and responsive across all device sizes.

The application is implemented as a Progressive Web Application (PWA) in order to provide a functional mobile interface without maintaining a separate codebase. This enables users to install the platform directly onto their device and allows the web application to access specialised device hardware, such as the camera for answer sheet capture. By maintaining a single codebase for both desktop and mobile, the platform ensures that all business logic remains consistent while the interface adapts to the user's current context and device capabilities.

4.7 DEPLOYMENT AND INFRASTRUCTURE

The deployment and infrastructure of EduPro were designed around three principles: environment consistency across development and production, automated quality enforcement on every code change, and safe delivery to a self-hosted production server. Docker, GitHub Actions, and a structured branching workflow were the primary tools used to achieve this.

4.7.1 INFRASTRUCTURE AND CONTAINERIZATION

The entire platform is containerised using Docker. This decision was driven by the diversity of the technology stack as it all needs to run together in a reproducible way. Docker Compose was chosen over more complex tools because the expected load is well within the capacity of a single server.

Two Compose configurations are maintained to serve different needs: during development, a lighter configuration allows each developer to start only the services relevant to their current work, keeping local environments fast and uncluttered; in production, a single unified configuration brings up all six services together: the application database, a dedicated Keycloak database, the Keycloak identity provider, a schema migration service, the FastAPI backend, and the Nginx reverse proxy.

Nginx sits at the entry point of the stack, terminating Transport Layer Security (TLS) and routing both `/api` and `/auth` under the same domain. This means the browser communicates with a single host, which eliminates cross-origin issues without any additional configuration. Behind Nginx, the migration service runs before the API starts: a `depends_on: service_completed_successfully` condition ensures the `API` container cannot boot if the database schema is not up to date, preventing the application from starting against a mismatched schema. Exam images are stored in a named Docker volume shared between the `API` container and the host, so they survive container restarts and deployments.

This infrastructure supported the full development lifecycle of the project. Six versioned releases were published on GitHub over the course of development, with `v1.1.1` as the latest stable version. Each release represents a stable, tested snapshot of the platform delivered to production through the automated Continuous Deployment (CD) pipeline, as shown in Figure 4.57.

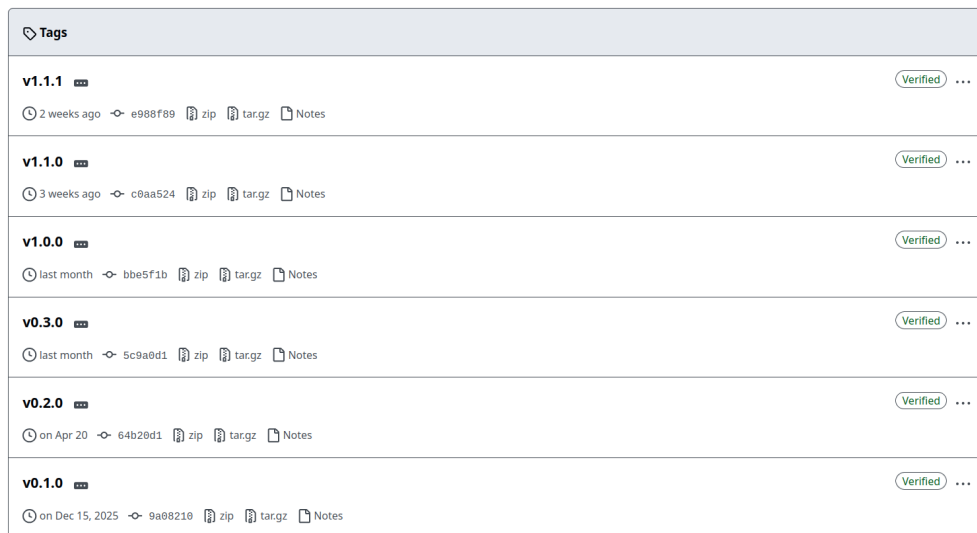


FIGURE 4.57 – GitHub tags page showing the six versioned releases of EduPro.

4.7.2 DEVELOPMENT WORKFLOW AND BRANCH PROTECTION

All development follows a feature-branch workflow. New code is written in a dedicated branch, and changes are introduced through pull requests targeting the `dev` branch, which is periodically promoted to `main`. Both `dev` and `main` are protected: force pushes are blocked, and a minimum of two approvals from team members is required before a pull request can be merged. Every pull request follows a structured template covering a description of the change, testing evidence, and a quality checklist that the author must confirm before requesting review. By the time the project reached its final milestone, 145 pull requests had been merged, as illustrated in Figure 4.58.

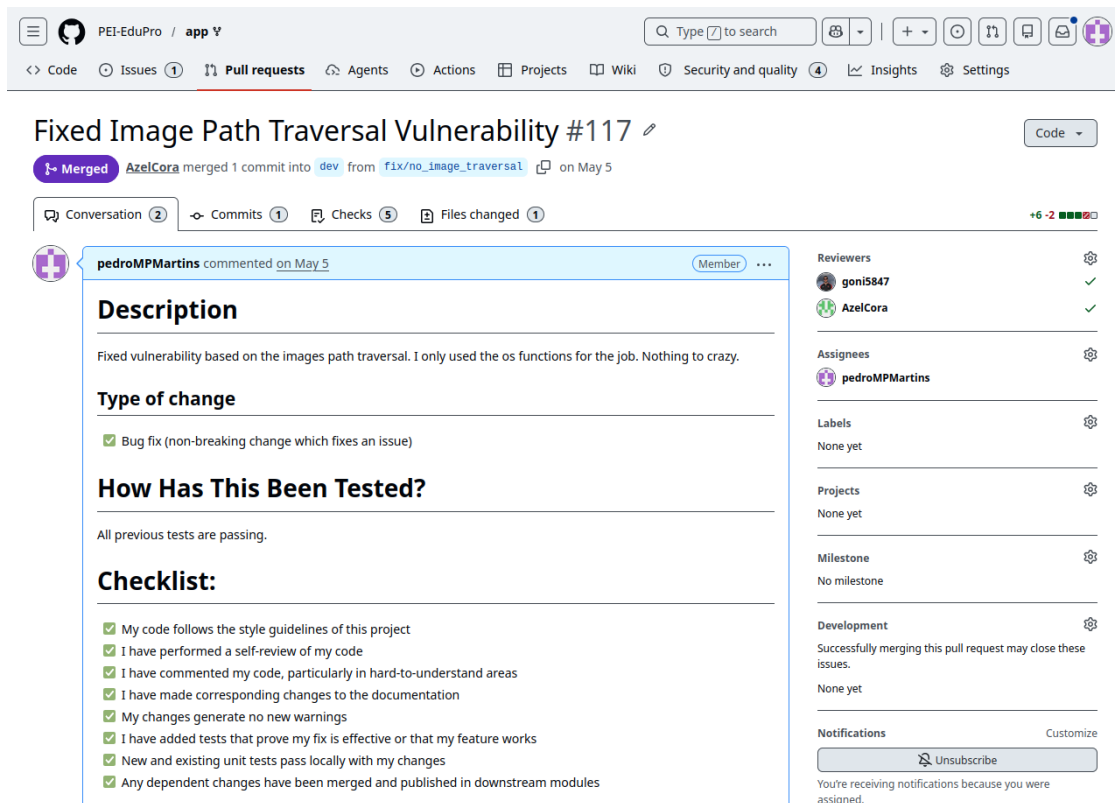


FIGURE 4.58 – GitHub Pull Request with completed checklist, two approvals, and all checks passing.

4.7.3 CI/CD PIPELINE

The Continuous Integration (CI) pipeline is triggered on every push and pull request to `dev` and `main`. It runs on GitHub-hosted runners and begins by building all Docker images to confirm the build configuration is valid. The migration service is then executed against the CI database and `alembic check` is run: if any unapplied migration exists, the pipeline fails immediately, preventing schema drift from reaching the protected branches. With the environment validated, the full test suite runs and coverage data is collected. Finally, SonarCloud analyses the results and enforces the quality gate before the change can be merged.

All third-party GitHub Actions are pinned to specific commit SHAs rather than mutable version tags, guarding against supply chain attacks where a compromised action could inject malicious code into the pipeline.

The CD pipeline runs on a self-hosted Linux runner installed directly on the production server, triggered exclusively on pushes to `main`. Running the deployment job on the server itself means no SSH keys or deployment credentials ever need to leave the machine. The deploy rebuilds only the changed images and restarts the affected containers with no manual intervention. Figure 4.59 shows all four checks passing on the main branch after a successful merge.

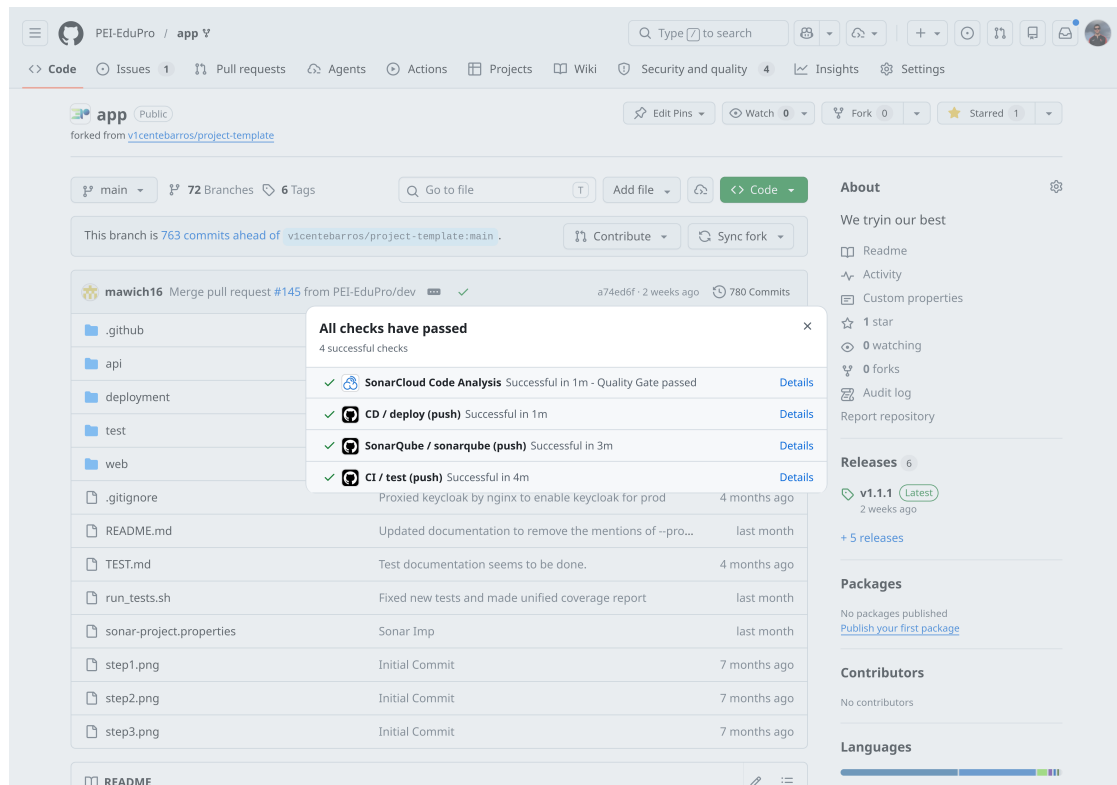


FIGURE 4.59 – Main branch with all four automated checks passing: [CI](#), [CD](#), [SonarQube](#), and [SonarCloud](#).

4.7.4 TEST VALIDATION STRATEGY

The test suite is split into two layers with distinct responsibilities. Unit tests verify individual functions and models in isolation using an in-memory configuration, providing fast feedback during development. Integration tests cover the interaction between the [API](#) and its live dependencies, PostgreSQL and Keycloak, and validate complete workflows such as exam generation, student association, and grade export.

To prevent interference with any running development instance, the integration tests spin up a fully isolated Docker environment using a dedicated Compose project (`edupro-test`), with the database on port 5433 and Keycloak on port 8081. A `trap` on the test script ensures this environment is torn down and all volumes are removed regardless of whether the tests pass or fail. Coverage results from both layers are merged into a single `coverage.xml` report and submitted to SonarCloud, which currently reports 66% line coverage across approximately 5k coverable lines, as shown in [Figure 4.60](#).

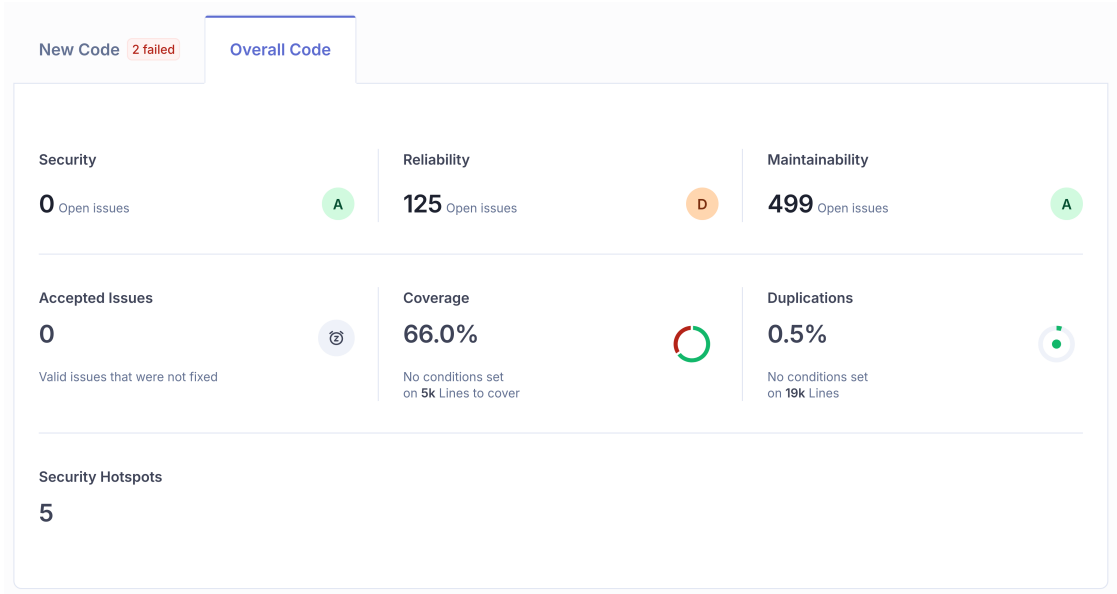


FIGURE 4.60 – SonarCloud overall code analysis: 0 security issues, 66% coverage, 0.5% duplication.

4.8 SECURITY

The security of the EduPro platform is implemented through a defense-in-depth strategy that addresses risks at the identity, data, and infrastructure levels. By combining centralized authentication, strict input validation, and infrastructure-level traffic control, the system ensures the integrity of academic data and the availability of services. This section details the specific mechanisms employed to protect the platform against common vulnerabilities and unauthorized access.

4.8.1 AUTHENTICATION AND AUTHORIZATION

The platform offloads identity management to Keycloak, an industry-standard OpenID Connect provider. This design choice is inherently more secure than a custom implementation because user credentials are never stored or processed by the application database, reducing the surface area for credential theft. Keycloak handles complex security flows such as token rotation, session management, and multi-factor authentication that adhere to modern security standards. The [API](#) utilizes JSON Web Token ([JWT](#)) signed by Keycloak, allowing every request to be independently verified for authenticity without a database lookup. Every request sent to the backend must be accompanied by a Bearer token, and the system performs a live query to the Keycloak Admin [API](#) to fetch the user’s current group memberships and roles. This validation and authorization flow is illustrated in [Figure 4.61](#).

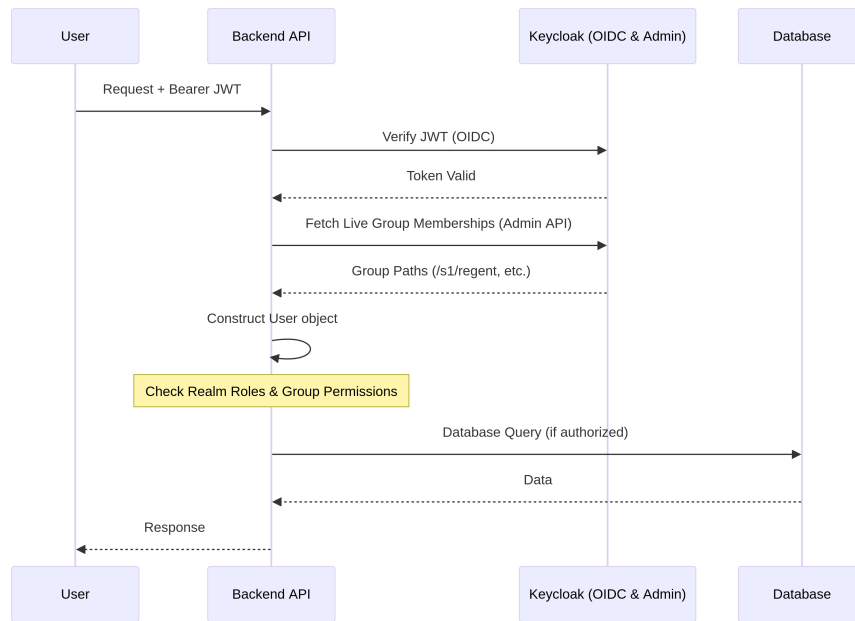


FIGURE 4.61 – Authentication and Authorisation Sequence Flow

This mechanism prevents the stale token problem, ensuring that revoked permissions take effect immediately without requiring the user to re-authenticate or wait for token expiration. Following the verification and retrieval of memberships, the system constructs an internal user object that encapsulates the user’s identity, providing a fine-grained permission framework based on global realm roles and hierarchical subject-specific groups.

4.8.2 DATA VALIDATION AND RATE LIMITING

To protect against malformed data and resource exhaustion, the system implements strict validation and traffic control. The [API](#) utilizes Pydantic for all request schemas, which automatically validates data types, ranges, and formats before the data reaches the business logic.

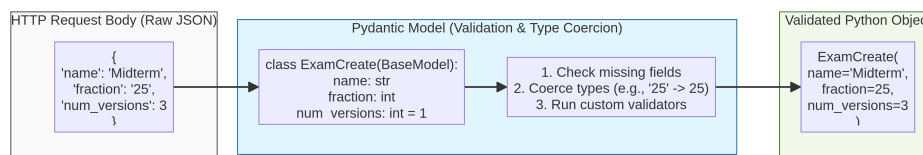


FIGURE 4.62 – Data validation and mapping using Pydantic schemas.

This layer prevents mass assignment vulnerabilities by explicitly defining which fields can be updated through the [API](#). At the infrastructure level, the Nginx reverse proxy enforces a limit of ten requests per second per IP address. This protects the system from brute-force attacks and denial of service attempts, with a burst buffer to allow for legitimate loading of complex dashboards. Additionally, specific endpoints such as [OMR](#) image uploads have capped request body sizes to prevent memory exhaustion during the processing of large files.

4.8.3 INJECTION PREVENTION

The platform implements multi-layered strategies to mitigate various injection vectors. By utilizing SQLAlchemy and SQLModel, the system ensures all database queries are parameterized, meaning that user input is never concatenated directly into Structured Query Language (SQL) strings. Since the system generates PDF documents from user-provided question text, it is also protected against LaTeX injection through an escaping layer that sanitizes special characters such as underscores, ampersands, and percent signs in student and exam names. For XML imports, the system uses the defusedxml library, which replaces standard XML parsers with safe alternatives that disable entity expansion and Document Type Definition (DTD) processing to prevent external entity attacks.

4.8.4 SECURITY HARDENING AND FIXES

During the development and auditing phase, several targeted security fixes were applied to further harden the platform. Log statements were audited to ensure that potentially malicious user-provided strings are sanitized or excluded from log outputs, preventing log-based injection attacks. The LaTeX compilation subprocess is isolated within a unique temporary directory, ensuring that filesystem artifacts are automatically cleaned up and preventing cross-request data leakage. Finally, the reverse proxy is configured to serve standard security headers that prevent content type sniffing, clickjacking, and cross-site scripting, ensuring a secure communication channel between the client and the server.

4.9 SUMMARY

This chapter provided a detailed exploration of the technical realization of the EduPro platform, illustrating the synergy between its various architectural layers and functional modules. The transition from the initial definition of the structural blueprint to the automated generation of randomized booklets established the foundation for a robust assessment lifecycle. By implementing a centralized state machine that governs every phase, from live session coordination to final grade distribution, the system ensures that administrative workflows remain validated and consistent, even when handling complex exceptions such as late image submissions.

The evolution of the OMR engine, particularly the transition from a traditional pixel-density approach to a convolutional neural network, highlights the project's commitment to robustness and performance. This deep learning approach, combined with sophisticated image normalization and perspective correction techniques, allows the platform to handle the inherent variability of physical classroom environments. Complementing these backend innovations, the web application provides a unified interface for professors, while the specialized PWA mobile interfaces empower vigilants with high-resolution hardware access and real-time association

capabilities.

Finally, the deployment and infrastructure strategy ensures that this complex ecosystem remains scalable and maintainable. The use of containerization, coupled with a rigorous [CI/CD](#) pipeline and static code analysis, provides a high level of confidence in the platform's stability and security. Through this dual-layered testing strategy and automated validation process, the EduPro platform is well-positioned to serve as a reliable tool for academic assessment. The following chapters will present an evaluation of the system's performance and discuss the results achieved during testing and real-world simulation.

5

CHAPTER

RESULTS AND DISCUSSION

This chapter presents the evaluation of the EduPro platform against the requirements established in Chapter 3. The system was assessed across three dimensions: real-world deployment, performance under load, and usability. The results demonstrate that the platform meets its defined non-functional requirements and was successfully validated in a production environment.

5.1 DEPLOYMENT

The EduPro platform was deployed in two stages. During the development and evaluation phase, the system was hosted within the university's internal network, where it was used by real users for usability testing and functional validation. This controlled environment allowed the team to observe the system under realistic academic conditions while maintaining close oversight.

Following the completion of the evaluation cycle, the platform was made publicly accessible. The system is currently live and exposed to the internet, with the [PWA](#) available for installation on mobile devices. This is particularly relevant for the vigilant workflow, where professors and teaching assistants use their smartphones to scan [QR](#) codes and photograph answer sheets during exam sessions. The [PWA](#) installation provides a near-native mobile experience without requiring distribution through an app store.

5.2 PERFORMANCE TESTING

Performance testing was conducted using K6 library to validate the non-functional requirement that the system must support a minimum of 50 concurrent users without significant latency or service interruption. Two load scenarios were tested, with results presented in Table 5.1.

TABLE 5.1 – K6 load testing results for two virtual user scenarios.

Scenario	Requests	Failed	Req/s	Avg	p90	p95
0–50 VUs	1215	0	9.9	55.08ms	95.49ms	137.07ms
0–200 VUs	1959	3	9.2	55.04ms	130.52ms	150.63ms

At 50 concurrent virtual users, the system processed all requests without a single failure, with a median response time of 43.52ms and a p95 of 137.07ms. These results confirm that the non-functional performance requirement is met.

At 200 VUs, only 3 failures were recorded out of 1959 requests, which is well within acceptable margins given that the defined requirement is 50 concurrent users. The average response time remained stable at 55ms across both scenarios, demonstrating that the system does not degrade significantly under increased load within the expected operational range.

During the preparation of these tests, an N+1 query problem was identified and resolved in the [API](#), which had been causing unnecessary database round-trips and contributing to latency spikes under load.

5.3 USABILITY EVALUATION

Usability was evaluated using the [SUS](#), a standardised questionnaire that produces a score between 0 and 100, where scores above 80 are considered good and above 90 excellent.

5.3.1 METHODOLOGY

Two rounds of usability testing were conducted. The first round took place with a group of 12 participants representing a diverse range of software literacy levels. Participants were asked to complete a set of tasks covering the Manager and Regent user flows, including creating a curricular unit, managing the question bank, and generating an exam. Following this round, the usability issues identified were addressed in subsequent development sprints. A second round was then conducted with 15 participants to evaluate both the impact of those improvements and the newly implemented system features that complete the full system. In addition to the tasks from the first round, participants evaluated the newly implemented workflows: downloading generated exams, scanning [QR](#) codes to associate students, conducting exam sessions, capturing answer sheets for automatic correction, reviewing and validating corrections, and exporting grades.

5.3.2 RESULTS

The first round produced a **SUS** score of **79.4**, placing the system just below the “good” threshold. Participants were generally able to complete tasks without assistance, but several issues were noted. **UI** elements were perceived as too small, particularly the breadcrumb navigation and the logout button. Certain colour combinations also lacked sufficient contrast against the white background, affecting readability. Some users felt disoriented after creating a topic, lacking clear guidance on the next step in the workflow.

These findings were addressed through targeted **UI** improvements, including increased element sizing, improved colour contrast, and the addition of contextual guidance cues in the question bank workflow.

The second round produced a **SUS** score of **87.3**, surpassing the target threshold of 80. Individual scores ranged from 55 to 100, with the majority above 80.

5.4 DISCUSSION

The results presented in this chapter demonstrate that EduPro meets its core non-functional requirements. The performance tests confirm that the system handles the required 50 concurrent users with zero failures and sub-150ms p95 response times. The initial **SUS** score of 79.4, while slightly below the target of 80, reflected the state of the system prior to the usability improvements. The second evaluation round confirmed that the target was reached, with a final score of 87.3.

The successful deployment within the university network, followed by the transition to a publicly accessible production environment, demonstrates that the platform is not only a functional prototype but a system ready for real-world academic use.

CHAPTER 6

CONCLUSION

This chapter summarises the work carried out throughout the project, reflecting on the level to which the stated goals were achieved, the lessons learned during development, and the directions that would most benefit future iterations of the platform.

6.1 MAIN RESULTS

EduPro was designed to address the fragmentation and labour-intensity of academic assessment management, and the platform delivered on that premise. At its core lies a structured, topic-organised question bank that supports Moodle XML import, allowing professors to migrate existing content into the system without manual re-entry and to build on it incrementally over successive semesters.

Built on top of that foundation, the exam generation engine translates a configurable selection of questions into multiple unique, typeset [PDF](#) variants. Each sheet carries an embedded [QR](#) code that uniquely identifies the exam variant and, once associated, the student; a mechanism that underpins the entire correction workflow without requiring any manual bookkeeping.

The correction itself is handled by an automated grading pipeline that combines classical image processing with a trained classifier to reduce answer sheet grading to a review-and-validate step. Rather than removing the professor from the loop entirely, the pipeline allows for human oversight where needed, while eliminating part of the repetitive mechanical work. Once corrections are validated, the platform can automatically generate per-student grade reports and dispatch them by email, closing the assessment cycle without further manual intervention.

Access control across the three platform roles (manager, regent, and professor) is enforced through Keycloak, keeping permission management outside the application logic; the platform was then deployed to a publicly accessible production environment, confirming EduPro as a live system beyond a controlled prototype.

6.2 RETROSPECTIVE

The project followed an iterative development approach, which allowed requirements to be refined as the complexity of each subsystem became clearer. The early investment in a well-defined data model (the Subject–Topic–Question hierarchy) created value throughout the implementation, providing a stable foundation for both the API and the frontend.

A challenging aspect of the project was the answer sheet correction pipeline. Achieving reliable cell extraction under variable scan and photograph quality required significant experimentation with pre-processing parameters and model training data. In hindsight, establishing a fixed, standardised scan protocol earlier in the project would have simplified this considerably.

6.3 FUTURE WORK

The platform is functional end to end, but several refinements could further enhance its institutional value. One immediate improvement would be the integration of a guided mobile capture assistant to provide real-time framing and quality feedback, which would streamline the answer sheet correction pipeline for photograph based submissions. Additionally, extending the correction engine to support short answer and written responses through Optical Character Recognition (OCR) or structured fields would broaden the platform’s applicability to a wider range of assessment types.

Future iterations would also benefit from a closer integration with institutional ecosystems. Connecting the platform to learning management systems like Moodle would allow validated grades to be synchronized automatically, eliminating the final manual bookkeeping step. Furthermore, developing an analytics dashboard to aggregate performance data per topic would provide educators with actionable insights into the difficulty calibration of their question banks over time. Finally, conducting validation trials in the production environment with real academic staff and students would provide the empirical evidence needed to confirm the platform’s effectiveness beyond controlled testing conditions.

REFERENCES

- [1] B. J. Mandernach and R. Holbeck, “Teaching online: Where do faculty spend their time?” *Online Journal of Distance Learning Administration*, vol. 19, no. 4, 2016. [Online]. Available: http://www.westga.edu/~distance/ojdla/winter194/mandernach_holbeck194.html.
- [2] B. Chen, M. West, and C. Zilles, “How much randomization is needed to deter collaborative cheating on asynchronous exams?” In *Proceedings of the Fifth Annual ACM Conference on Learning at Scale*, ACM, 2018. DOI: [10.1145/3231644.3231664](https://doi.org/10.1145/3231644.3231664).

ANNEX A — USE CASE DIAGRAM

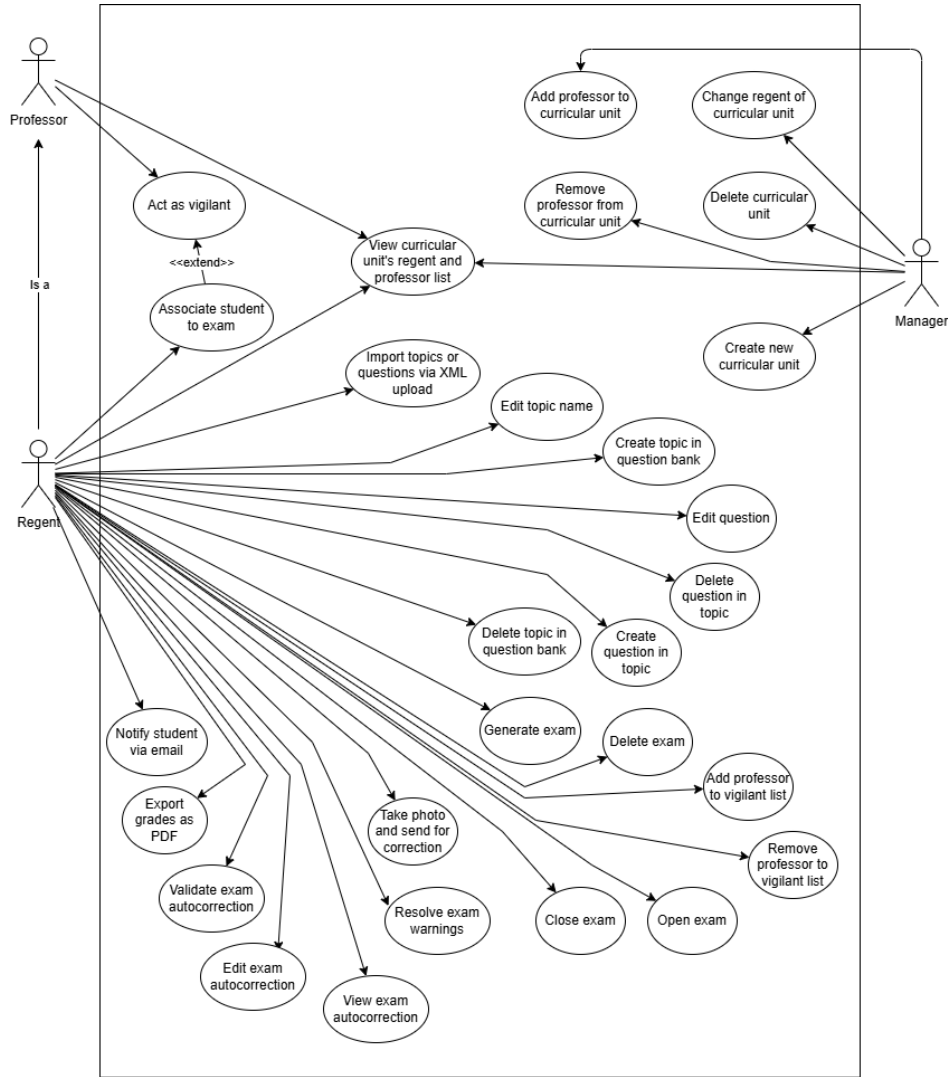


FIGURE 1 – Use Case Diagram for EduPro.